



# **PixelProse: An Explainable AI Image Caption Generator using Vision–Language Transformers**

**LY Innovation-Based Major Project-B Report**

**Submitted in partial fulfillment of the requirements of the Degree of  
Bachelor of Technology in Computer Engineering**

by

**Kartik Verma**

**Dhir Thakar**

**Kushal Soni**

LY COMPS-B

**Supervisor**

**Prof. Pradnya Bangale**



Department of Computer Engineering

K. J. Somaiya Institute of Technology

**An Autonomous Institute permanently affiliated to University of Mumbai**

**Ayurvihar, Sion, Mumbai -400022**

**2026-27**



# **PixelProse: An Explainable AI Image Caption Generator using Vision–Language Transformers**

**LY Innovation-Based Major Project-B Report**

**Submitted in partial fulfillment of the requirements of the Degree of  
Bachelor of Technology in Computer Engineering**

by

**Kartik Verma (Roll No. \_\_\_\_\_)**

**Dhir Thakar (Roll No. \_\_\_\_\_)**

**Kushal Soni (Roll No. \_\_\_\_\_)**

LY COMPS-B

**Supervisor**

**Prof. Pradnya Bangale**



Department of Computer Engineering

K. J. Somaiya Institute of Technology

**An Autonomous Institute permanently affiliated to University of Mumbai**

**Ayurvihar, Sion, Mumbai -400022**

**2026-27**



## CERTIFICATE



*This is to certify that the project entitled “**PixelProse: An Explainable AI Image Caption Generator using Vision–Language Transformers**” is bonafide work of **Kartik Verma, Dhir Thakar, Kushal Soni** submitted to the University of Mumbai in partial fulfillment of the requirement in Major Project, for the award of the degree of “**Bachelors of Technology**” in “**Computer Engineering**”.*

---

**Prof. Pradnya Bangale**

Project Guide

Department of Computer Engineering

---

**Dr. Sarita Ambadekar,**

Head, Department of Computer  
Engineering

---

**Dr. Vivek Sunnapwar**

Principal KJSIT

Place: Sion, Mumbai-400022

Date:

## PROJECT APPROVAL FOR L. Y.

This project report entitled “**PixelProse: An Explainable AI Image Caption Generator using Vision–Language Transformers**” by

Kartik Verma (Roll No. \_\_\_\_\_)

Dhir Thakar (Roll No. \_\_\_\_\_)

Kushal Soni (Roll No. \_\_\_\_\_)

is an approved Last Year Innovation-Based Major Project **in Computer Engineering.**

### Examiners:

1. \_\_\_\_\_  
Name and Signature  
**External Examiner**

2. \_\_\_\_\_  
Name and Signature  
**Internal Examiner**

Place: Sion, Mumbai-400022

Date:

## DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

---

Kartik Verma

---

Dhir Thakar

---

Kushal Soni

Date:

## ACKNOWLEDGEMENT

Before presenting our LY Major Project work entitled “**PixelProse: An Explainable AI Image Caption Generator using Vision–Language Transformers**”, we would like to convey our sincere thanks to the people who guided us throughout the course for this project work.

First, we would like to express our immense gratitude towards our Project Guide **Prof. Pradnya Bangale** for the constant encouragement, support, guidance, and mentoring at the ongoing stages of the project and report.

We would like to express our sincere thanks to our H.O.D. **Dr. Sarita Ambadekar**, for the encouragement, co-operation, and suggestions progressing stages of the report.

We would like to express our sincere thanks to our beloved Principal **Dr. Vivek Sunnapwar** for providing various facilities to carry out this project.

Finally, we would like to thank all the teaching and non-teaching staff of the college, and our friends, for their moral support rendered during the course of the reported work, and for their direct and indirect involvement in the completion of our report work, which made our endeavor fruitful.

Kartik Verma  
Dhir Thakar  
Kushal Soni

Place: Sion, Mumbai-400022

Date:

# ABSTRACT

Image captioning, the automatic generation of a natural-language description for an image, lies at the intersection of computer vision and natural language processing and has important applications in accessibility, image search and assistive technology. Although modern vision–language transformers achieve high benchmark scores, practical captioning tools usually return a single unverified sentence, offer no insight into how it was produced, and often depend on paid cloud services.

This project presents PixelProse, an explainable image caption generator built on the BLIP (Bootstrapping Language-Image Pre-training) family of models. A Vision Transformer encodes the image once into 577 patch embeddings, and an image-grounded text decoder generates several candidate captions using beam search and, optionally, nucleus sampling. Each candidate receives a language-model confidence computed from its token probabilities. An independent image–text matching model then verifies and re-ranks the candidates, so the selected caption is both fluent and grounded in the picture. To make the model transparent, Grad-CAM applied to the cross-attention layers produces a heatmap for every word, showing the image region the word relied on. A visual question-answering model lets users ask free-form questions about the image, and a matching tool scores user-written descriptions.

The system is implemented with PyTorch, Hugging Face Transformers and FastAPI, and it is delivered as an animated, responsive and accessible web application with five input methods, batch processing, history, text-to-speech and a documented REST API and command-line interface. It runs entirely on a laptop CPU. On a hand-annotated set of 44 images with 132 reference captions, the default configuration achieves a CIDEr-D of 1.945, a BLEU-4 of 0.408 and a 92 % mean image–text match in about 3.9 seconds per image, while beam search improves CIDEr-D by 7.6 % over a greedy baseline. PixelProse demonstrates that accurate, verifiable and explainable image captioning is achievable privately on commodity hardware.

**Keywords:** *Image captioning, Vision Transformer, BLIP, beam search, image–text matching, Grad-CAM, explainable AI, visual question answering, CIDEr.*

# CONTENTS

| Chapter No. |      | TITLE                                | Page No. |
|-------------|------|--------------------------------------|----------|
|             |      | <b>LIST OF FIGURES</b>               | viii     |
|             |      | <b>LIST OF TABLES</b>                | ix       |
|             |      | <b>LIST OF ABBREVIATIONS</b>         | x        |
| <b>1</b>    |      | <b>INTRODUCTION</b>                  | 1        |
|             | 1.1  | Introduction                         | 1        |
|             | 1.2  | Problem Definition                   | 3        |
|             | 1.3  | Aim and Objectives                   | 4        |
|             | 1.4  | Organization of the Report           | 5        |
| <b>2</b>    |      | <b>REVIEW OF LITERATURE</b>          | 6        |
|             | 2.1  | Literature Survey                    | 6        |
|             | 2.2  | Research Gaps                        | 10       |
| <b>3</b>    |      | <b>REQUIREMENT SPECIFICATION</b>     | 12       |
|             | 3.1  | Introduction                         | 12       |
|             | 3.2  | Hardware Requirements                | 12       |
|             | 3.3  | Software Requirements                | 13       |
|             | 3.4  | Feasibility Study                    | 13       |
|             | 3.5  | Cost Estimation                      | 14       |
|             | 3.6  | Functional Requirements              | 15       |
|             | 3.7  | Non-Functional Requirements          | 16       |
| <b>4</b>    |      | <b>PROJECT ANALYSIS &amp; DESIGN</b> | 17       |
|             | 4.1  | Introduction                         | 17       |
|             | 4.2  | UML Diagrams                         | 17       |
|             | 4.3  | System Architecture                  | 26       |
|             | 4.4  | API Design                           | 27       |
|             | 4.5  | User Interface Design                | 27       |
|             | 4.6  | Data Design                          | 28       |
| <b>5</b>    |      | <b>METHODOLOGY</b>                   | 29       |
|             | 5.1  | Introduction                         | 29       |
|             | 5.2  | System Block Diagram                 | 29       |
|             | 5.3  | Image Pre-processing                 | 30       |
|             | 5.4  | Vision Transformer Encoder           | 31       |
|             | 5.5  | Image-Grounded Text Decoder          | 32       |
|             | 5.6  | BLIP Pre-training Objectives         | 33       |
|             | 5.7  | Decoding Strategies                  | 33       |
|             | 5.8  | Candidate Scoring and Re-ranking     | 35       |
|             | 5.9  | Explainability with Grad-CAM         | 36       |
|             | 5.10 | Visual Question Answering            | 37       |
|             | 5.11 | Post-processing                      | 37       |
|             | 5.12 | Evaluation Metrics                   | 38       |
|             | 5.13 | Overall Algorithm                    | 39       |
| <b>6</b>    |      | <b>IMPLEMENTATION DETAILS</b>        | 40       |

| <b>Chapter No.</b> | <b>TITLE</b>                                                          | <b>Page No.</b> |
|--------------------|-----------------------------------------------------------------------|-----------------|
|                    | 6.1 Introduction                                                      | 40              |
|                    | 6.2 Development Environment and Technology Stack                      | 40              |
|                    | 6.3 Project Structure                                                 | 40              |
|                    | 6.4 Backend Implementation                                            | 41              |
|                    | 6.5 Frontend Implementation                                           | 43              |
|                    | 6.6 System Implementation Screenshots                                 | 44              |
|                    | 6.7 Testing                                                           | 49              |
|                    | 6.8 Deployment                                                        | 50              |
| <b>7</b>           | <b>RESULT ANALYSIS</b>                                                | 51              |
|                    | 7.1 Introduction                                                      | 51              |
|                    | 7.2 Experimental Setup                                                | 51              |
|                    | 7.3 Performance Measures                                              | 51              |
|                    | 7.4 Qualitative Results                                               | 54              |
|                    | 7.5 Visual Question Answering Results                                 | 56              |
|                    | 7.6 Explainability Results                                            | 56              |
|                    | 7.7 Engineering Findings                                              | 57              |
|                    | 7.8 Comparative Analysis                                              | 57              |
|                    | 7.9 Limitations                                                       | 58              |
| <b>8</b>           | <b>CONCLUSION &amp; FUTURE SCOPE</b>                                  | 60              |
|                    | 8.1 Conclusion                                                        | 60              |
|                    | 8.2 Future Scope                                                      | 61              |
|                    | <b>REFERENCES</b>                                                     | 62              |
|                    | <b>PUBLISHED PAPERS</b>                                               | 65              |
|                    | <b>CERTIFICATES (Paper Publications / Conferences / Competitions)</b> | 66              |
|                    | <b>PLAGIARISM REPORT</b>                                              | 67              |

# LIST OF FIGURES

| Figure No. | Title                                                                                   | Page No. |
|------------|-----------------------------------------------------------------------------------------|----------|
| 1.1        | An example of automatic image captioning by PixelProse                                  | 2        |
| 2.1        | Evolution of image captioning approaches (2010–2023)                                    | 6        |
| 4.1        | Use-case diagram of PixelProse                                                          | 18       |
| 4.2        | Sequence diagram for caption generation                                                 | 19       |
| 4.3        | Sequence diagram for visual question answering                                          | 20       |
| 4.4        | Activity diagram of the captioning workflow                                             | 21       |
| 4.5        | State diagram of the Caption Studio interface                                           | 22       |
| 4.6        | Class diagram of the backend                                                            | 23       |
| 4.7        | Component diagram                                                                       | 24       |
| 4.8        | Deployment diagram                                                                      | 25       |
| 4.9        | Data flow diagram – Level 0 (context diagram)                                           | 25       |
| 4.10       | Data flow diagram – Level 1                                                             | 26       |
| 5.1        | System block diagram of PixelProse                                                      | 30       |
| 5.2        | The image is resized to 384×384 and split into 24×24 = 576 patches of 16×16 pixels      | 31       |
| 5.3        | Architecture of the BLIP captioning model (ViT encoder and image-grounded decoder)      | 32       |
| 5.4        | Beam search keeps the k best partial captions at every step (pruned paths shown dashed) | 34       |
| 5.5        | Grad-CAM pipeline for word-level explanations                                           | 37       |
| 6.1        | Home page (hero section) of PixelProse                                                  | 44       |
| 6.2        | Caption Studio with a generated caption and ranked candidates                           | 44       |
| 6.3        | Explain tab: attention heatmap and word-by-word confidence                              | 45       |
| 6.4        | Word-level heatmaps produced by Grad-CAM                                                | 46       |
| 6.5        | Ask tab (visual question answering) and Match tab (scoring user descriptions)           | 46       |
| 6.6        | Insights tab: keywords, palette, metadata, timing and alt-text                          | 47       |
| 6.7        | Batch mode with export to CSV and JSON                                                  | 47       |
| 6.8        | Responsive layout on a mobile phone                                                     | 48       |
| 6.9        | Interactive REST API documentation (Swagger UI)                                         | 48       |
| 6.10       | Command-line interface output                                                           | 49       |
| 7.1        | CIDEr-D of the evaluated configurations                                                 | 52       |
| 7.2        | BLEU-1...4 and ROUGE-L of the evaluated configurations                                  | 53       |
| 7.3        | Image–text match versus latency (bubble area $\propto$ CIDEr-D)                         | 53       |
| 7.4        | ROUGE-L by image category                                                               | 54       |
| 7.5        | Qualitative results of PixelProse (Balanced mode) with human references                 | 55       |

## LIST OF TABLES

| Table No. | Title                                                                         | Page No. |
|-----------|-------------------------------------------------------------------------------|----------|
| 2.1       | Summary of the literature survey                                              | 9        |
| 3.1       | Hardware requirements                                                         | 12       |
| 3.2       | Software requirements                                                         | 13       |
| 3.3       | Cost estimation                                                               | 15       |
| 3.4       | Functional requirements                                                       | 15       |
| 4.1       | REST API endpoints                                                            | 27       |
| 6.1       | Technology stack                                                              | 40       |
| 6.2       | Frontend JavaScript modules                                                   | 43       |
| 6.3       | Representative test cases                                                     | 49       |
| 7.1       | Experimental setup                                                            | 51       |
| 7.2       | Caption quality and speed of the evaluated configurations                     | 52       |
| 7.3       | Latency of each pipeline stage in Balanced mode (4-core CPU)                  | 54       |
| 7.4       | Captions produced by different configurations                                 | 55       |
| 7.5       | Sample visual question answering results                                      | 56       |
| 7.6       | Engineering issues found during testing                                       | 57       |
| 7.7       | Published results on MS-COCO (Karpathy test split) as reported by the authors | 57       |
| 7.8       | Feature comparison with existing captioning solutions                         | 58       |

## LIST OF ABBREVIATIONS

| Sr. No | Abbreviation | Description                                             |
|--------|--------------|---------------------------------------------------------|
| 1      | AI           | Artificial Intelligence                                 |
| 2      | API          | Application Programming Interface                       |
| 3      | ASGI         | Asynchronous Server Gateway Interface                   |
| 4      | BERT         | Bidirectional Encoder Representations from Transformers |
| 5      | BLEU         | Bilingual Evaluation Understudy                         |
| 6      | BLIP         | Bootstrapping Language-Image Pre-training               |
| 7      | CIDEr        | Consensus-based Image Description Evaluation            |
| 8      | CLI          | Command-Line Interface                                  |
| 9      | CNN          | Convolutional Neural Network                            |
| 10     | COCO         | Common Objects in Context (dataset)                     |
| 11     | CPU          | Central Processing Unit                                 |
| 12     | CUDA         | Compute Unified Device Architecture                     |
| 13     | DFD          | Data Flow Diagram                                       |
| 14     | GPU          | Graphics Processing Unit                                |
| 15     | Grad-CAM     | Gradient-weighted Class Activation Mapping              |
| 16     | HTML         | HyperText Markup Language                               |
| 17     | ITC          | Image-Text Contrastive (learning)                       |
| 18     | ITM          | Image-Text Matching                                     |
| 19     | JSON         | JavaScript Object Notation                              |
| 20     | LM           | Language Modelling                                      |
| 21     | LRU          | Least Recently Used (cache)                             |
| 22     | LSTM         | Long Short-Term Memory                                  |
| 23     | MHSA         | Multi-Head Self-Attention                               |
| 24     | MPS          | Metal Performance Shaders (Apple GPU backend)           |
| 25     | NLP          | Natural Language Processing                             |
| 26     | PPL          | Perplexity                                              |
| 27     | REST         | Representational State Transfer                         |
| 28     | RNN          | Recurrent Neural Network                                |
| 29     | ROUGE        | Recall-Oriented Understudy for Gisting Evaluation       |
| 30     | SHA          | Secure Hash Algorithm                                   |
| 31     | SSRF         | Server-Side Request Forgery                             |
| 32     | TF-IDF       | Term Frequency-Inverse Document Frequency               |
| 33     | UI           | User Interface                                          |
| 34     | UML          | Unified Modelling Language                              |
| 35     | URL          | Uniform Resource Locator                                |
| 36     | ViT          | Vision Transformer                                      |
| 37     | VLP          | Vision-Language Pre-training                            |
| 38     | VQA          | Visual Question Answering                               |
| 39     | WCAG         | Web Content Accessibility Guidelines                    |

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Computer vision and natural language processing have long developed as separate branches of Artificial Intelligence. Computer vision teaches machines to recognise objects, scenes and actions in pixels, while natural language processing teaches them to understand and produce human language. **Image captioning** sits exactly at the intersection of the two: given a photograph, a system must understand what is happening in the scene and then express that understanding as a fluent, grammatical sentence such as “*a woman sitting on a beach next to a dog*”. The task is deceptively simple for humans, yet it requires a machine to detect objects, infer their attributes and relationships, decide which details matter, and order words correctly, all in one pass. It is therefore regarded as one of the benchmark problems of multimodal artificial intelligence.

Early captioning systems relied on hand-written templates filled in by object detectors, or on retrieving the caption of the most similar image from a database. The deep-learning era replaced these with *encoder–decoder* networks: a Convolutional Neural Network (CNN) encoded the image into a vector and a Recurrent Neural Network (RNN) decoded the vector into words. Since 2017 the **Transformer** architecture, built entirely on the attention mechanism, has taken over both halves of the pipeline. Modern *vision–language pre-training* (VLP) models such as BLIP are first trained on hundreds of millions of image–text pairs from the web and then fine-tuned for captioning, reaching near-human scores on standard benchmarks.

This report presents **PixelProse**, an *explainable* image caption generator developed as our Last-Year Innovation-Based Major Project. PixelProse uses the BLIP family of vision–language transformers from Salesforce Research and wraps them in a complete, locally-running application. Beyond producing a caption, the system generates several candidate sentences and asks a second neural network to verify which one best matches the image. It visualises, word by word, the image regions the model relied on, answers free-form questions about the picture, and measures its own quality with standard research metrics. Everything is delivered through a modern, animated web interface and

a REST API, and it runs on an ordinary laptop CPU without any paid cloud service. Figure 1.1 shows a typical output of the system.



**Figure 1.1:** An example of automatic image captioning by PixelProse

### 1.1.1 Motivation

Our motivation comes from three directions. First, **accessibility**: according to the World Health Organization, at least 2.2 billion people live with a near or distance vision impairment [1]. Blind and low-vision users depend on screen readers, which in turn depend on *alternative text* (alt-text) describing each image. Surveys of the most popular websites repeatedly find that more than half of their home pages contain images with missing alt-text [2]. Automatic captioning can close much of this gap. Second, **information retrieval**: billions of photographs are uploaded every day, and search engines, digital libraries and photo applications need textual descriptions to index them. Third, **education and trust in AI**: deep networks are frequently criticised as black boxes. A captioning tool that can *explain* its output provides an excellent vehicle for studying and demonstrating how attention-based models actually reason.

### 1.1.2 Applications of Image Captioning

- **Assistive technology**: describing surroundings, documents and photographs to visually impaired users through text-to-speech.
- **Web accessibility compliance**: generating alt-text for websites and content-management systems (WCAG guidelines).
- **Search and indexing**: enabling text-based search over large, untagged image collections.

- **Social media and marketing:** suggesting captions, keywords and hashtags for posts.
- **Content moderation:** summarising the content of uploaded images for automatic or human review.
- **Robotics and autonomous systems:** letting robots and smart assistants describe what their cameras see.
- **Medical and scientific imaging:** drafting preliminary descriptions of scans or microscopy images (with domain-specific training).

## 1.2 Problem Definition

Formally, image captioning is the problem of learning a function that maps an image  $I$  to a sequence of words  $S = (w_1, w_2, \dots, w_n)$  that describes the image accurately and fluently. The model is trained to maximise the conditional probability  $P(S | I)$ . At inference time it must search for the most probable sentence among an exponentially large set of possible word sequences.

Although state-of-the-art models achieve impressive benchmark scores, several practical problems remain when they are used as real tools:

1. **Single, unverified output.** Typical tools return one caption with no indication of how reliable it is, even though models sometimes *hallucinate* objects that are not present.
2. **Lack of explainability.** Users cannot see *why* the model produced a particular word, which limits trust and makes errors hard to diagnose.
3. **No interaction.** A caption is a fixed summary; users cannot ask follow-up questions about details the caption omitted.
4. **Dependence on cloud services.** Commercial captioning APIs require an internet connection, charge per request and send private photographs to third-party servers.
5. **Poor usability for non-experts.** Research code is released as scripts and notebooks rather than as usable applications.
6. **Unmeasured quality.** Demonstration tools rarely report objective quality metrics, so different decoding choices cannot be compared.

The problem addressed by this project is therefore: **to design and implement an accurate, explainable, interactive and privacy-preserving image caption generator**

**that runs on commodity hardware, and to evaluate its quality objectively using standard captioning metrics.**

### **1.3 Aim and Objectives**

**Aim:** To develop *PixelProse*, a web-based image caption generator built on vision–language transformers that produces accurate natural-language descriptions of images, verifies and explains them, and supports interactive question answering, all running locally on a personal computer.

**Objectives:**

1. To study the evolution of image captioning from CNN–RNN models to vision–language transformers and select a suitable pre-trained model.
2. To build a caption-generation pipeline that encodes each image once and generates multiple candidate captions using beam search and nucleus sampling.
3. To estimate the confidence of each caption from token probabilities and to re-rank candidates with an independent image–text matching (ITM) model.
4. To make the model explainable through word-level Grad-CAM heatmaps over cross-attention and per-word probability visualisation.
5. To integrate visual question answering (VQA) and free-text description matching so users can interrogate the image.
6. To design an attractive, responsive and accessible web interface with multiple input methods (upload, drag-and-drop, paste, URL, webcam and samples).
7. To expose all functionality through a documented REST API and a command-line tool.
8. To implement the BLEU, ROUGE-L and CIDEr-D metrics and evaluate different decoding configurations on a hand-annotated test set.
9. To ensure the system runs on a CPU-only laptop with acceptable latency, and to document, test and present the work.

#### **1.3.1 Scope of the Project**

The project covers English-language captioning of still photographs and artworks using pre-trained BLIP models, with inference on CPU, CUDA GPUs or Apple Silicon. Training a captioning model from scratch is outside the scope, because it requires hundreds of millions of image–text pairs and weeks of GPU time. Instead the project demonstrates *transfer learning*: it builds a complete, engineered and evaluated system

around publicly released pre-trained weights. Video captioning, multilingual output and domain-specific fine-tuning are identified as future scope.

## **1.4 Organization of the Report**

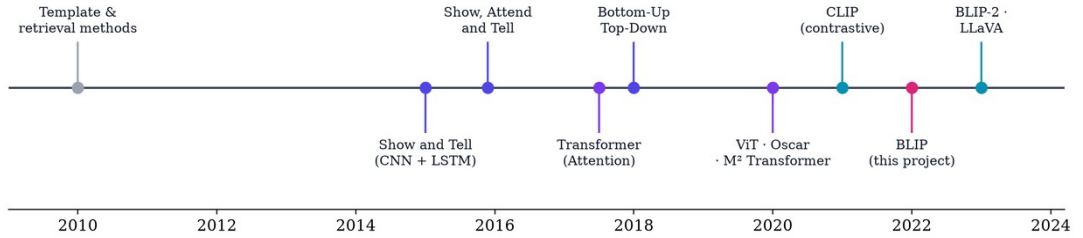
The rest of this report is organised as follows. **Chapter 2** surveys captioning and vision–language research and identifies research gaps. **Chapter 3** specifies the hardware, software, functional and non-functional requirements with the feasibility study and cost estimation. **Chapter 4** presents the UML diagrams, data flow diagrams, system architecture and API and interface design. **Chapter 5** explains the methodology with its mathematical formulation: the Vision Transformer and text decoder, decoding algorithms, candidate scoring and re-ranking, Grad-CAM explainability and the evaluation metrics. **Chapter 6** describes the implementation, screenshots and testing. **Chapter 7** analyses the results and compares the system with existing work, and **Chapter 8** concludes the report with future scope.

## CHAPTER 2

### REVIEW OF LITERATURE

#### 2.1 Literature Survey

**Survey strategy.** We began with two comprehensive surveys of deep-learning-based image captioning [3], [4] to understand the overall taxonomy of the field. From their references we followed the most-cited works in each generation of methods: template-based systems, CNN–RNN encoder–decoders, attention-based models, transformer models and large-scale vision–language pre-training. We searched Google Scholar, IEEE Xplore, the CVF Open Access repository and arXiv for “image captioning”, “vision-language pre-training”, “visual attention”, “visual question answering” and “caption evaluation metric”, and prioritised peer-reviewed papers from CVPR, ICCV, ECCV, ICML, NeurIPS, ICLR and ACL. For each paper we recorded the methodology, the dataset, the reported results and the limitations relevant to our goals of accuracy, explainability and deployability. The survey is presented in text in this section and in tabular form in Table 2.1. Figure 2.1 places the key works on a timeline.



**Figure 2.1:** Evolution of image captioning approaches (2010–2023)

##### 2.1.1 Template and Retrieval-Based Methods

The earliest captioning systems generated sentences by filling linguistic templates with the outputs of object, attribute and scene detectors. *Baby Talk* by Kulkarni et al. [5] detected objects and their spatial relations, built a conditional random field over them and rendered the most probable labelling through templates such as “*the [adjective], [object] is [preposition] the [object]*”. Retrieval-based approaches instead searched a large captioned database for visually similar images and transferred their captions. Both families produce grammatical but rigid sentences and cannot describe novel combinations of objects, which motivated the move to end-to-end neural generation.

### 2.1.2 CNN–RNN Encoder–Decoder Models

Vinyals et al. introduced *Show and Tell* [6], the Neural Image Caption (NIC) generator, which framed captioning as machine translation from pixels to words. A GoogLeNet CNN encodes the image into a single feature vector that initialises a Long Short-Term Memory (LSTM) decoder, and the whole network is trained end to end to maximise the likelihood of the reference caption. NIC achieved a BLEU-4 of 27.7 on MS-COCO [7] and demonstrated that a single neural network could outperform complex pipelines. In parallel, Karpathy and Fei-Fei [8] aligned image regions with sentence fragments using a multimodal embedding and generated region-level descriptions with an RNN; their train/validation/test split of COCO (the *Karpathy split*) became the standard benchmark. The main limitation of these models is the *information bottleneck*: compressing an entire scene into one vector loses the spatial detail needed to describe multiple objects.

### 2.1.3 Visual Attention

Xu et al. removed the bottleneck in *Show, Attend and Tell* [9]. The CNN produces a  $14 \times 14$  grid of feature vectors, and at every time step the LSTM computes attention weights over the grid, attending to the region relevant to the next word. The paper proposed both deterministic “soft” and stochastic “hard” attention and visualised the attention maps, an early form of explainability that directly inspired our heatmap feature. Lu et al. [10] observed that non-visual words such as “the” or “of” should not rely on the image at all and proposed *adaptive attention* with a visual sentinel that lets the model decide when to look. Rennie et al. [11] introduced *self-critical sequence training* (SCST), a reinforcement-learning technique that directly optimises the non-differentiable CIDEr metric and substantially improved scores.

Anderson et al. proposed *Bottom-Up and Top-Down attention* [12]. A Faster R-CNN object detector (bottom-up) proposes salient object regions, and a two-layer LSTM (top-down) attends over these object-level features instead of a uniform grid. The approach reached a BLEU-4 of 36.3 and a CIDEr of 120.1 on COCO and also won the 2017 VQA challenge, showing that the same visual representation serves captioning and question answering. Its drawbacks are the expensive detector and the dependence on the detector's object vocabulary.

#### 2.1.4 Transformer-Based Captioning

Vaswani et al. introduced the *Transformer* [13], an architecture based entirely on multi-head self-attention without recurrence. It captures long-range dependencies and trains in parallel. Cornia et al. applied it to captioning in the *Meshed-Memory Transformer* (M<sup>2</sup>) [14], whose encoder learns a-priori knowledge through memory slots and whose decoder attends to all encoder layers through a meshed connectivity. It achieved a BLEU-4 of 39.1 and a CIDEr of 131.2. Dosovitskiy et al. then showed with the *Vision Transformer* (ViT) [15] that a pure transformer applied to sequences of 16×16 image patches matches or beats CNNs when pre-trained on sufficient data. The ViT is the image encoder used by BLIP and hence by PixelProse.

#### 2.1.5 Vision–Language Pre-training

The next leap came from pre-training on large image–text corpora before fine-tuning. *Oscar* [16] used object tags detected in the image as anchor points to align visual regions with words during pre-training, reaching a BLEU-4 of 41.7 and a CIDEr of 140.0 on COCO. Radford et al.'s *CLIP* [17] trained separate image and text encoders with a *contrastive* objective on 400 million web pairs, learning a joint embedding space with remarkable zero-shot transfer. However, CLIP has no decoder and cannot generate text. *ALBEF* [18] proposed aligning image and text representations with a contrastive loss before fusing them with cross-attention, together with momentum distillation to cope with noisy web data. It also demonstrated Grad-CAM visualisations on the cross-attention layers, the technique we adopt for explainability.

Li et al.'s **BLIP** [19], the foundation of this project, unifies understanding and generation in a *Multimodal mixture of Encoder–Decoder* (MED). A ViT image encoder is shared by three heads trained jointly: an image–text contrastive (ITC) loss that aligns unimodal embeddings, an image–text matching (ITM) loss that classifies whether a pair matches using cross-attention, and a language-modelling (LM) loss that trains an image-grounded decoder to generate text. BLIP also introduced **CapFilt**: a captioner generates synthetic captions for web images and a filter removes noisy original and synthetic captions, bootstrapping a cleaner 129-million-image dataset. The ViT-L model reaches a BLEU-4 of 40.4 and a CIDEr of 136.7 on COCO. Crucially for us, the ITM head provides a built-in way to *verify* a caption, and the model is small enough (446M parameters) to run on a laptop CPU.

Later work scales the idea further. *GIT* [20] simplified the architecture to one image encoder and one text decoder trained on up to 0.8 billion pairs. *BLIP-2* [21] bridges a frozen image encoder and a frozen large language model with a lightweight *Querying Transformer*, achieving a BLEU-4 of 43.7 and a CIDEr of 145.8 on COCO. These models, however, need billions of parameters and generally a GPU, which conflicts with our goal of running on student hardware.

### 2.1.6 Explainability, Question Answering and Evaluation

Selvaraju et al.'s **Grad-CAM** [22] produces class-discriminative localisation maps by weighting feature maps with the gradients of the output score, and it applies to any differentiable architecture. Antol et al. defined **Visual Question Answering (VQA)** [23]: answering free-form natural-language questions about an image, a task that tests fine-grained understanding. BLIP provides a VQA model built on the same backbone. For evaluation, **BLEU** [24] measures modified n-gram precision with a brevity penalty, **ROUGE-L** [25] measures the longest common subsequence with the references, **CIDEr** [26] weights n-grams by TF-IDF to reward informative words and correlates best with human consensus, and **SPICE** [27] compares scene graphs. On decoding, Holtzman et al. [28] showed that beam search tends to produce generic, repetitive text and proposed *nucleus (top-p) sampling*, which motivates our combination of both strategies.

### 2.1.7 Summary of Literature

**Table 2.1:** Summary of the literature survey

| Sr. | Authors (Year)            | Method                                     | Dataset            | Key result                                | Limitation                          |
|-----|---------------------------|--------------------------------------------|--------------------|-------------------------------------------|-------------------------------------|
| 1   | Kulkarni et al. (2011)    | Detectors + CRF + templates                | PASCAL             | Grammatical template sentences            | Rigid, cannot describe novel scenes |
| 2   | Vinyals et al. (2015)     | CNN encoder + LSTM decoder (NIC)           | MS-COCO, Flickr    | BLEU-4 27.7 on COCO                       | Single-vector bottleneck            |
| 3   | Karpathy & Fei-Fei (2015) | Region-word alignment + RNN                | Flickr8K/30K, COCO | Region-level descriptions; standard split | Weak global coherence               |
| 4   | Xu et al. (2015)          | Soft / hard visual attention over CNN grid | Flickr, COCO       | BLEU-4 24.3 (soft); attention maps        | Grid features, LSTM decoding        |
| 5   | Lu et al. (2017)          | Adaptive attention, visual sentinel        | Flickr30K, COCO    | Better grounding of visual words          | Still LSTM-based                    |
| 6   | Rennie et al. (2017)      | Self-critical RL on CIDEr                  | COCO               | Large CIDEr gains                         | Optimises metric, may hurt fluency  |

| Sr. | Authors (Year)               | Method                                   | Dataset                   | Key result                           | Limitation                         |
|-----|------------------------------|------------------------------------------|---------------------------|--------------------------------------|------------------------------------|
| 7   | Anderson et al. (2018)       | Bottom-up detector + top-down attention  | COCO, VQA v2              | BLEU-4 36.3, CIDEr 120.1             | Costly object detector             |
| 8   | Vaswani et al. (2017)        | Transformer (self-attention only)        | WMT translation           | State of the art, parallel training  | Text-only                          |
| 9   | Cornia et al. (2020)         | Meshed-memory transformer                | COCO                      | BLEU-4 39.1, CIDEr 131.2             | Detector features, no pre-training |
| 10  | Dosovitskiy et al. (2021)    | Vision Transformer on 16×16 patches      | ImageNet, JFT-300M        | Matches CNNs at scale                | Needs large pre-training           |
| 11  | Li et al. – Oscar (2020)     | VLP with object-tag anchors              | 6.5M pairs; COCO          | BLEU-4 41.7, CIDEr 140.0             | Relies on detector tags            |
| 12  | Radford et al. – CLIP (2021) | Contrastive image–text encoders          | 400M web pairs            | Strong zero-shot transfer            | Cannot generate captions           |
| 13  | Li et al. – ALBEF (2021)     | Align-before-fuse, momentum distillation | 14M pairs                 | Strong retrieval; Grad-CAM grounding | Not a captioner by default         |
| 14  | Li et al. – BLIP (2022)      | MED (ITC+ITM+LM) with CapFilt            | 129M pairs; COCO          | BLEU-4 40.4, CIDEr 136.7 (ViT-L)     | Single caption, no explanation UI  |
| 15  | Wang et al. – GIT (2022)     | Unified generative image-to-text         | 0.8B pairs                | Strong captioning & VQA              | Very large; GPU required           |
| 16  | Li et al. – BLIP-2 (2023)    | Frozen ViT + Q-Former + frozen LLM       | 129M pairs; COCO          | BLEU-4 43.7, CIDEr 145.8             | Billions of parameters             |
| 17  | Selvaraju et al. (2017)      | Gradient-weighted class activation maps  | ImageNet, VQA             | Visual explanations for any CNN      | Coarse resolution                  |
| 18  | Antol et al. (2015)          | Visual question answering task           | VQA (COCO images)         | Benchmark for image QA               | Answer priors / biases             |
| 19  | Vedantam et al. (2015)       | CIDEr consensus metric                   | PASCAL-50S, ABSTRAC T-50S | Best agreement with humans           | Needs multiple references          |
| 20  | Holtzman et al. (2020)       | Nucleus (top-p) sampling                 | Open-ended text           | Less repetitive, more natural text   | Randomness, can drift              |

## 2.2 Research Gaps

The literature shows rapid progress in benchmark accuracy, but our review identified the following gaps between research models and usable, trustworthy captioning tools:

- **Explainability is rarely delivered to end users.** Attention and Grad-CAM visualisations exist in papers [9], [22], [18], but publicly available captioning tools return plain text only. Users cannot inspect *which region* produced *which word*.
- **Captions are not verified.** Generative models are known to hallucinate objects, yet common pipelines return the single highest-likelihood sentence. The ITM

head learned during BLIP pre-training [19] is normally discarded at inference instead of being used to cross-check the generated caption.

- **Decoding choices are under-explored in practice.** Papers usually report one decoding setting. The trade-off between beam search, sampling and re-ranking [28] is seldom exposed to or measured for users.
- **Heavy compute and cloud dependence.** The strongest recent models [20], [21] require GPUs or paid APIs, which excludes students, offline settings and privacy-sensitive users.
- **Lack of interactivity.** A single caption summarises an image; question answering and free-text matching are usually separate research demos rather than parts of one tool.
- **Limited accessibility features.** Few tools integrate alt-text generation, speech output and keyboard accessibility aligned with WCAG [29].

PixelProse addresses these gaps by combining multi-candidate decoding, ITM-based verification, word-level Grad-CAM explanations, VQA and description matching in a single application that runs locally on a CPU, and by evaluating its decoding strategies with standard metrics.

## CHAPTER 3

### REQUIREMENT SPECIFICATION

#### 3.1 Introduction

A requirement specification describes *what* the system must do and *under which constraints*, before the design describes *how*. For PixelProse the requirements were derived from the objectives in Section 1.3, the research gaps in Section 2.2 and the practical context of the project: it must run on the laptops students already own, it must work without paid services, and it must be demonstrable live in a classroom. This chapter lists the hardware and software requirements, the functional and non-functional requirements, the feasibility study and the cost estimation.

#### 3.2 Hardware Requirements

Deep-learning inference is dominated by matrix multiplications, so processor speed and memory determine the user experience. PixelProse loads four BLIP networks totalling about 1.25 billion parameters. In 32-bit floating point this needs roughly 5 GB of RAM, which drives the memory requirement in Table 3.1. A GPU is optional: on CPU a caption takes about four seconds, while a CUDA GPU brings it well under one second.

**Table 3.1:** Hardware requirements

| Component         | Minimum                                                    | Recommended                                                         |
|-------------------|------------------------------------------------------------|---------------------------------------------------------------------|
| Processor         | 4-core x86-64 or Apple Silicon CPU (2015 or newer)         | 6–8 core CPU (Intel Core i5/i7 10th gen+, AMD Ryzen 5/7, Apple M1+) |
| Memory (RAM)      | 8 GB (with VQA disabled or base model)                     | 16 GB                                                               |
| Storage           | 6 GB free (models $\approx$ 4 GB + environment)            | 10 GB free SSD                                                      |
| GPU               | Not required                                               | NVIDIA GPU with $\geq$ 4 GB VRAM (CUDA) or Apple M-series (MPS)     |
| Display           | 1366 $\times$ 768                                          | 1920 $\times$ 1080 or projector                                     |
| Camera (optional) | Any webcam for live capture                                | HD webcam                                                           |
| Network           | Internet for the one-time model download ( $\approx$ 4 GB) | Wi-Fi for LAN demos on phones                                       |

### 3.3 Software Requirements

**Table 3.2:** Software requirements

| Category         | Software / Library                        | Version                 | Purpose                                       |
|------------------|-------------------------------------------|-------------------------|-----------------------------------------------|
| Operating system | Windows 10/11, macOS 12+, Ubuntu 20.04+   | —                       | Host platform                                 |
| Language         | Python                                    | 3.10 – 3.12             | Backend, AI pipeline, scripts                 |
| Deep learning    | PyTorch                                   | $\geq 2.2$              | Tensor computation, autograd for Grad-CAM     |
| Models           | Hugging Face Transformers                 | 4.56 – 4.x              | BLIP models, tokenizers, generation           |
| Model hub        | huggingface_hub, safetensors              | $\geq 0.23, \geq 0.4$   | Downloading and loading weights               |
| Web framework    | FastAPI + Uvicorn                         | $\geq 0.110, \geq 0.27$ | REST API, OpenAPI docs, ASGI server           |
| Uploads          | python-multipart                          | $\geq 0.0.9$            | Multipart form parsing                        |
| Imaging          | Pillow, NumPy                             | $\geq 10.0, \geq 1.26$  | Decoding, EXIF, palette, metadata             |
| Frontend         | HTML5, CSS3, JavaScript (ES2020 modules)  | —                       | User interface, Canvas, Web Speech API        |
| Browser          | Chrome, Edge, Firefox or Safari           | Latest                  | Running the web interface                     |
| Testing          | pytest, httpx                             | $\geq 8.0, \geq 0.27$   | Automated unit and API tests                  |
| Tools            | Git/GitHub, VS Code, PlantUML, Matplotlib | —                       | Version control, editing, diagrams and charts |

### 3.4 Feasibility Study

#### 3.4.1 Technical Feasibility

All required components are mature and open source. BLIP models are publicly available through the Hugging Face Hub under the BSD-3-Clause licence, and PyTorch runs them efficiently on CPUs. Our measurements (Chapter 7) show a caption in about 3.9 seconds on a 4-core CPU with the default configuration, which is acceptable for interactive use. The team had prior experience with Python and web development, and the attention mechanism and transformers are covered in the Advanced AI syllabus. The project is therefore technically feasible.

#### 3.4.2 Economic Feasibility

The project uses only free and open-source software and runs on the students' existing laptops. No GPU purchase, cloud computing credits or paid API subscriptions are required. The only recurring cost is electricity during development, which is negligible. Compared with commercial captioning APIs that charge per image, PixelProse has zero marginal cost per caption. The project is economically feasible.

### 3.4.3 Operational Feasibility

The system is operated through a web browser with a familiar drag-and-drop interface. One-click setup and start scripts for Windows (setup.bat, start.bat) and macOS/Linux (setup.sh, start.sh) remove the need for command-line knowledge. The interface provides immediate visual feedback, tooltips and keyboard shortcuts, so users need no training. The project is operationally feasible.

### 3.4.4 Schedule Feasibility

Using pre-trained models avoids the months of training that building a model from scratch would take. The work was divided into literature study, backend pipeline, interface, evaluation, and documentation and presentation, which were executed partly in parallel by the three team members within the semester. The project is feasible within the academic schedule.

### 3.4.5 Legal and Ethical Feasibility

The BLIP weights are released under the BSD-3-Clause licence, which permits academic and commercial use with attribution. The test images come from public test-image collections of open-source projects and are used only for evaluation. Because images never leave the user's machine, the system avoids the privacy concerns of cloud APIs. We acknowledge that models trained on web data can reflect social biases; this is discussed as a limitation and future work.

## 3.5 Cost Estimation

We estimate the development effort with the **Basic COCOMO** model [30] in *organic* mode, which suits small teams working on familiar problems. The implemented system contains about 4.6 thousand non-blank lines of code: 1,493 lines of Python (application and scripts), 180 lines of tests, 1,775 lines of JavaScript, 625 lines of CSS and 562 lines of HTML. With KLOC = 4.6:

$$E = a \cdot (\text{KLOC})^b = 2.4 \times (4.6)^{1.05} \approx 12.0 \text{ person-months} \quad (3.1)$$

$$D = c \cdot (E)^d = 2.5 \times (12.0)^{0.38} \approx 6.4 \text{ months} \quad (3.2)$$

$$P = E / D = 12.0 / 6.4 \approx 1.9 \text{ persons} \quad (3.3)$$

The model therefore predicts about two people working for six to seven months, which is consistent with our three-member team working part-time alongside coursework

during the semester. Table 3.3 converts the effort into a notional monetary cost, assuming an intern-level rate of ₹25,000 per person-month.

**Table 3.3:** Cost estimation

| Item                           | Basis                                             | Cost (₹)           |
|--------------------------------|---------------------------------------------------|--------------------|
| Development effort             | 12.0 person-months $\times$ ₹25,000               | 3,00,000           |
| Hardware                       | Existing student laptops (no purchase)            | 0                  |
| Software and libraries         | Open source (PyTorch, Transformers, FastAPI, ...) | 0                  |
| Pre-trained models             | Salesforce BLIP, BSD-3-Clause                     | 0                  |
| Cloud / API usage              | Not required (runs locally)                       | 0                  |
| Internet (model download)      | $\approx$ 4 GB one-time on existing connection    | $\approx$ 50       |
| Printing and binding of report | Estimated                                         | $\approx$ 1,500    |
| Total (notional)               |                                                   | $\approx$ 3,01,550 |

The actual cash expenditure is only the printing cost; the development-effort figure represents the value of the work produced.

### 3.6 Functional Requirements

Functional requirements specify the behaviour the system must provide. They are listed in Table 3.4 with an identifier used for traceability to the test cases in Chapter 6.

**Table 3.4:** Functional requirements

| ID    | Requirement                                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1  | The system shall accept an image by file upload, drag-and-drop, clipboard paste, public URL, webcam capture or selection from bundled samples.   |
| FR-2  | The system shall validate the image (format, size $\leq$ 15 MB, decodability) and reject invalid input with a clear message.                     |
| FR-3  | The system shall generate an English caption for the image using a pre-trained BLIP model.                                                       |
| FR-4  | The system shall generate multiple candidate captions and display them with their scores.                                                        |
| FR-5  | The system shall compute a confidence score for every caption and a probability for every word.                                                  |
| FR-6  | The system shall re-rank candidates using an image–text matching model and display the match probability.                                        |
| FR-7  | The system shall display a word-level attention heatmap over the image for any word of the caption.                                              |
| FR-8  | The system shall answer free-form questions about the image (visual question answering).                                                         |
| FR-9  | The system shall score user-written descriptions against the image.                                                                              |
| FR-10 | The system shall let the user choose the model (large/base), caption style, candidate count, length, beam width, creativity and a guided prefix. |
| FR-11 | The system shall provide keywords, hashtags, a colour palette, image metadata, stage timings and accessible alt-text HTML.                       |
| FR-12 | The system shall read captions aloud and export a shareable caption card.                                                                        |
| FR-13 | The system shall caption up to 12 images in batch mode and export the results as CSV or JSON.                                                    |

| ID    | Requirement                                                                                      |
|-------|--------------------------------------------------------------------------------------------------|
| FR-14 | The system shall store the last 30 results in the browser history and allow them to be restored. |
| FR-15 | The system shall expose all features through a documented REST API and a command-line interface. |

### 3.7 Non-Functional Requirements

- **Performance:** a caption with the default settings shall be produced in under 5 seconds on a 4-core laptop CPU and under 1 second on a GPU. Repeated requests for the same image and settings shall be served from cache almost instantly.
- **Accuracy:** the default configuration shall achieve a CIDEr-D above 1.9 and an image–text match above 90 % on the bundled evaluation set (Chapter 7).
- **Usability:** a first-time user shall be able to caption an image in under 30 seconds without instructions. The interface shall work on screens from 390 px (phone) to 1920 px wide.
- **Accessibility:** keyboard navigation, visible focus states, ARIA labels, reduced-motion support and text-to-speech shall be provided in line with WCAG 2.1 [29].
- **Reliability:** a failure in one optional model (ITM or VQA) shall not stop captioning; errors shall be reported as readable messages.
- **Security and privacy:** images shall be processed locally. URL fetching shall block private network addresses (SSRF protection), uploads shall be size-limited and all user text shall be escaped before rendering.
- **Portability:** the system shall run on Windows, macOS and Linux with CPU, CUDA or Apple MPS acceleration, selected automatically.
- **Maintainability:** the code shall be modular (separate services for captioning, matching, VQA and image handling), documented and covered by automated tests.

## CHAPTER 4

# PROJECT ANALYSIS & DESIGN

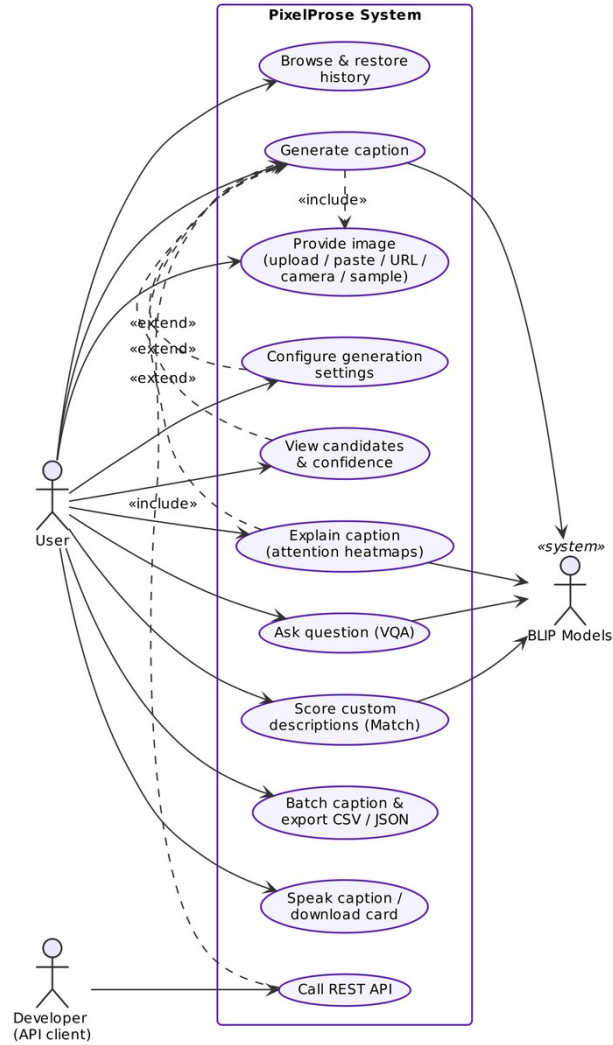
### 4.1 Introduction

This chapter translates the requirements of Chapter 3 into a design. We follow the Unified Modelling Language (UML) to describe the system from several complementary viewpoints. The *use-case* view captures what users can do. *Sequence* and *activity* diagrams capture the dynamic behaviour. The *state* diagram captures the life cycle of the interface. *Class*, *component* and *deployment* diagrams capture the static structure and the runtime environment. Data Flow Diagrams (DFDs) show how information moves through the system. The chapter closes with the layered system architecture, the REST API design and the user-interface design.

### 4.2 UML Diagrams

#### 4.2.1 Use-Case Diagram

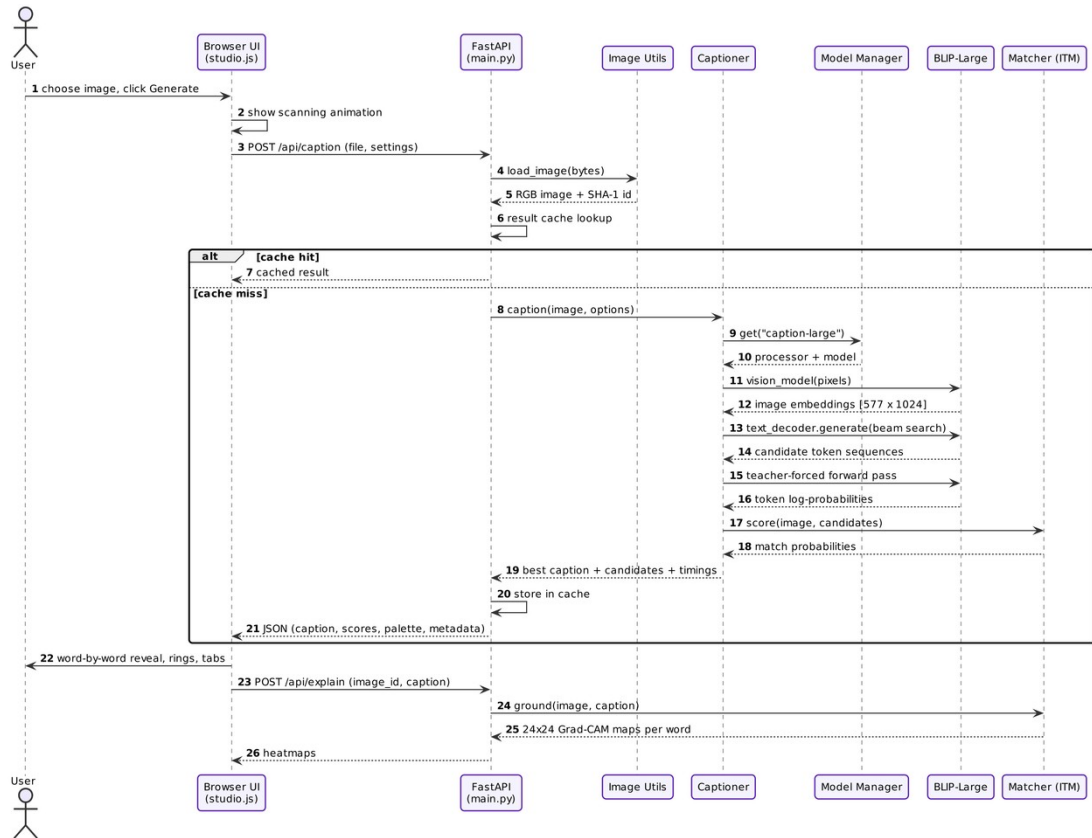
Figure 4.1 shows the use cases of PixelProse. The primary actor is the **User**, who provides an image through one of five input methods and generates a caption. *Generate caption* includes *Provide image*, and it is extended by *Configure generation settings*, *View candidates and confidence* and *Explain caption*, which are optional behaviours available after a caption exists. The user can also ask questions about the image, score custom descriptions, run batch captioning with export, listen to or download the caption and restore past results from history. A second actor, the **Developer**, accesses the same functionality programmatically through the REST API. The **BLIP models** appear as a supporting system actor that performs inference for captioning, explanation, question answering and matching.



**Figure 4.1:** Use-case diagram of PixelProse

#### 4.2.2 Sequence Diagram – Caption Generation

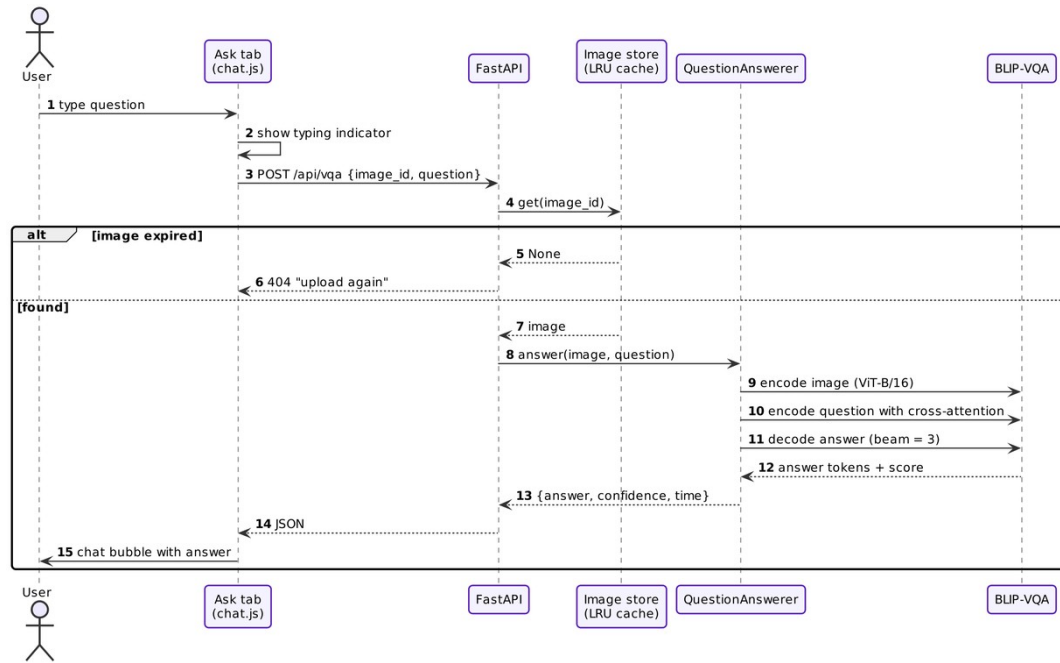
Figure 4.2 details the message flow for the central use case. After the user clicks *Generate*, the browser starts the scanning animation and posts the image with the chosen settings to `/api/caption`. The API validates the image with the image utilities and computes its SHA-1 identifier, which serves as a cache key. On a cache hit the stored result is returned immediately. Otherwise the Captioner obtains the BLIP-large model from the Model Manager, encodes the image once with the Vision Transformer, generates candidate sequences with beam search, computes token log-probabilities with one teacher-forced pass and asks the Matcher for image–text match probabilities. The best caption with its candidates and timings is cached and returned. The browser then reveals the caption word by word and, in the background, requests Grad-CAM heatmaps from `/api/explain` so that they are ready when the user opens the Explain tab.



**Figure 4.2:** Sequence diagram for caption generation

#### 4.2.3 Sequence Diagram – Visual Question Answering

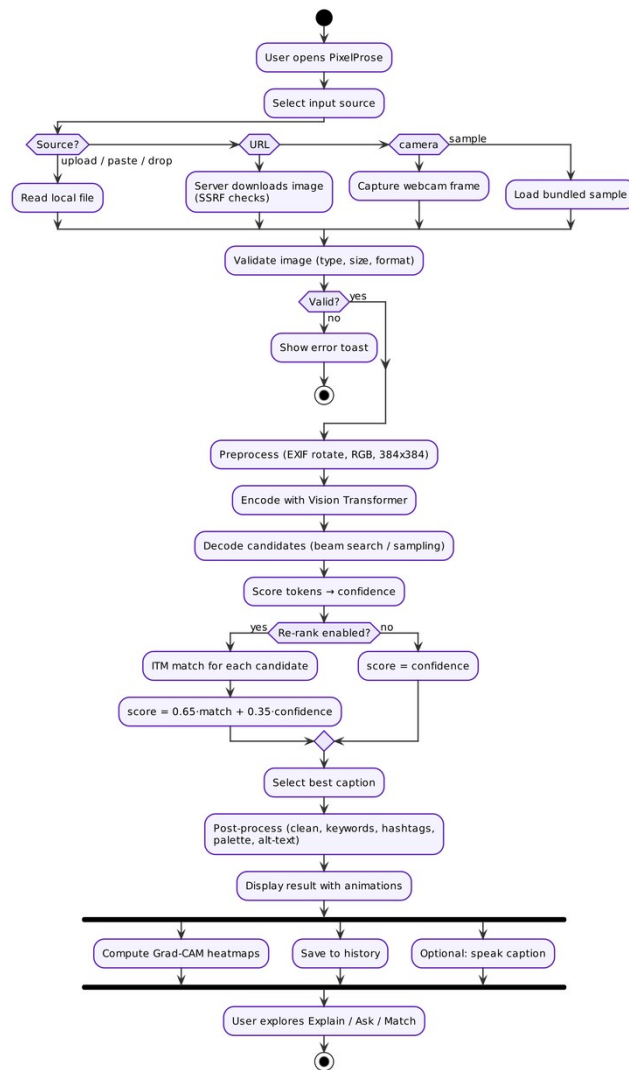
Figure 4.3 shows how a question is answered. Because the image was already uploaded during captioning, the browser sends only the short image identifier and the question. The API retrieves the image from the in-memory LRU store; if it has expired, a 404 response tells the user to upload it again. Otherwise the QuestionAnswerer runs BLIP-VQA: it encodes the image with ViT-B/16, encodes the question with cross-attention to the image, and decodes a short answer with beam search. The answer and its confidence appear as a chat bubble.



**Figure 4.3:** Sequence diagram for visual question answering

#### 4.2.4 Activity Diagram

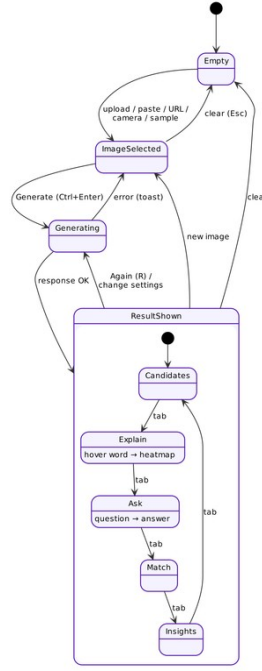
The activity diagram in Figure 4.4 captures the complete workflow, including decisions. The input source determines how the image is obtained. Validation either stops the flow with an error toast or continues to pre-processing. After decoding and scoring, a decision node checks whether re-ranking is enabled and computes the final score accordingly. After the best caption is displayed, a fork runs three activities in parallel: computing the Grad-CAM heatmaps, saving the result to history and optionally speaking the caption. The user may then explore the Explain, Ask and Match features.



**Figure 4.4:** Activity diagram of the captioning workflow

#### 4.2.5 State Diagram

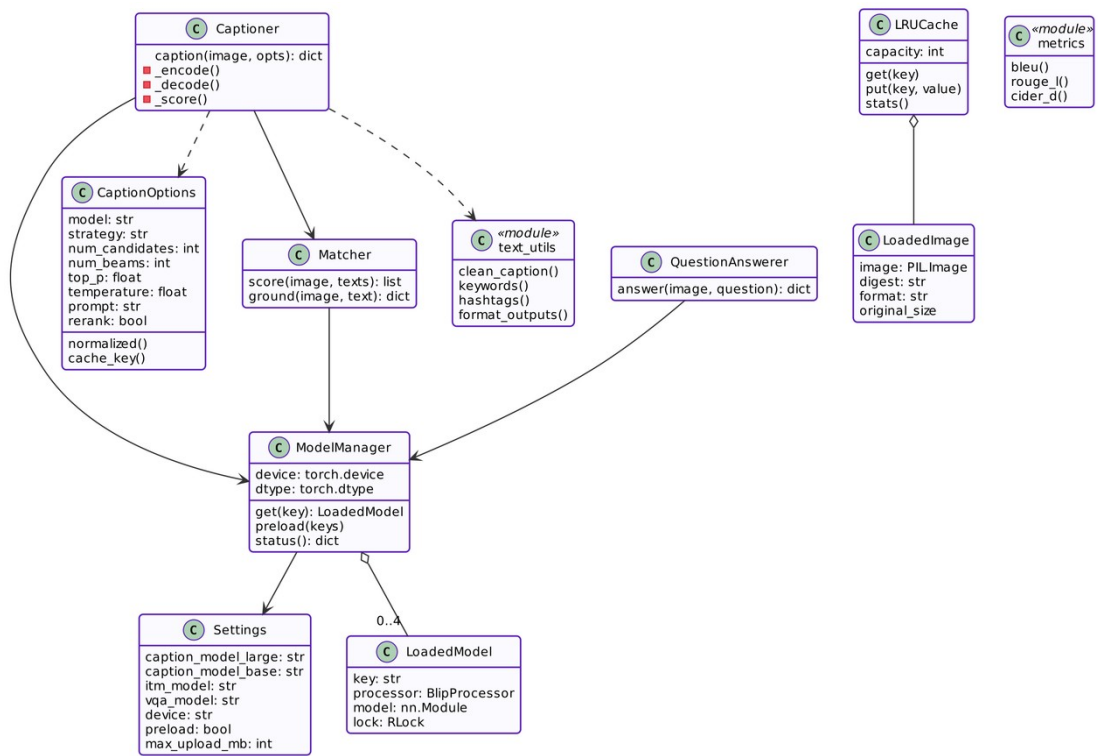
Figure 4.5 models the life cycle of the Caption Studio interface. It starts in the *Empty* state. Providing an image moves it to *ImageSelected*, and generating moves it to *Generating*, during which the scanning animation and pipeline checklist are shown. A successful response leads to *ResultShown*, a composite state whose sub-states are the five result tabs (Candidates, Explain, Ask, Match and Insights). An error returns the interface to *ImageSelected* with a message. From *ResultShown* the user can regenerate, choose a new image or clear everything.



**Figure 4.5:** State diagram of the Caption Studio interface

#### 4.2.6 Class Diagram

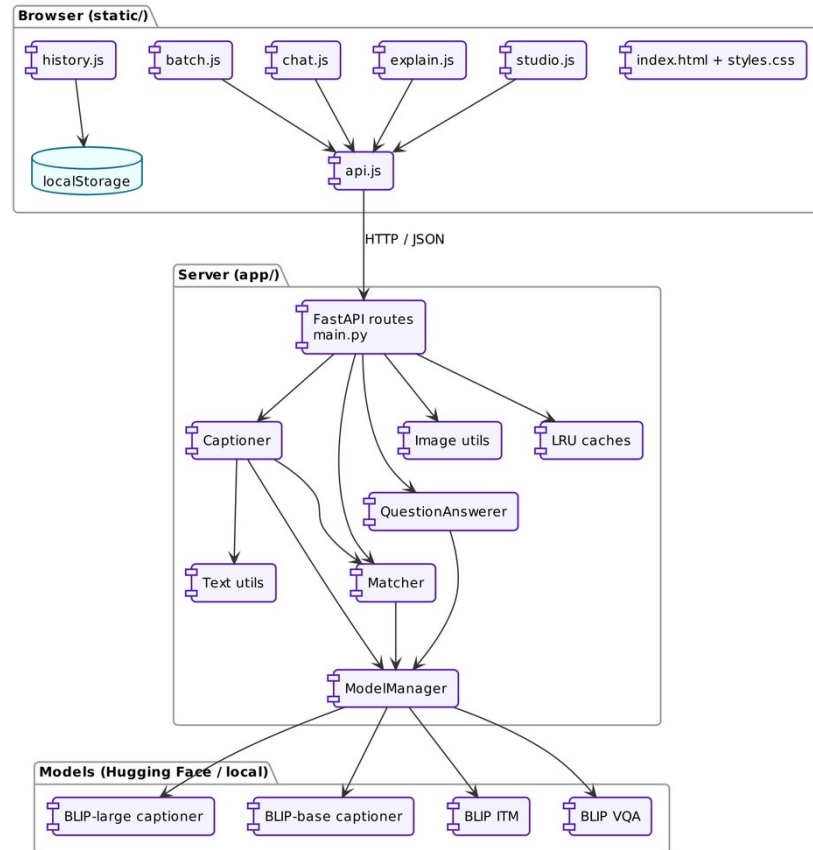
Figure 4.6 presents the main backend classes. **ModelManager** owns up to four **LoadedModel** objects (processor, network and an inference lock) and depends on the **Settings** read from environment variables. **Captioner** uses the **ModelManager**, delegates re-ranking to the **Matcher** and receives its parameters as a **CaptionOptions** value object, whose `normalized()` method clamps user input to safe ranges and whose `cache_key()` identifies deterministic requests. **QuestionAnswerer** uses the **ModelManager** for VQA. **LRUCache** stores **LoadedImage** objects and results. The `text_utils` and `metrics` modules provide stateless functions for caption cleaning and evaluation.



**Figure 4.6:** Class diagram of the backend

### 4.2.7 Component Diagram

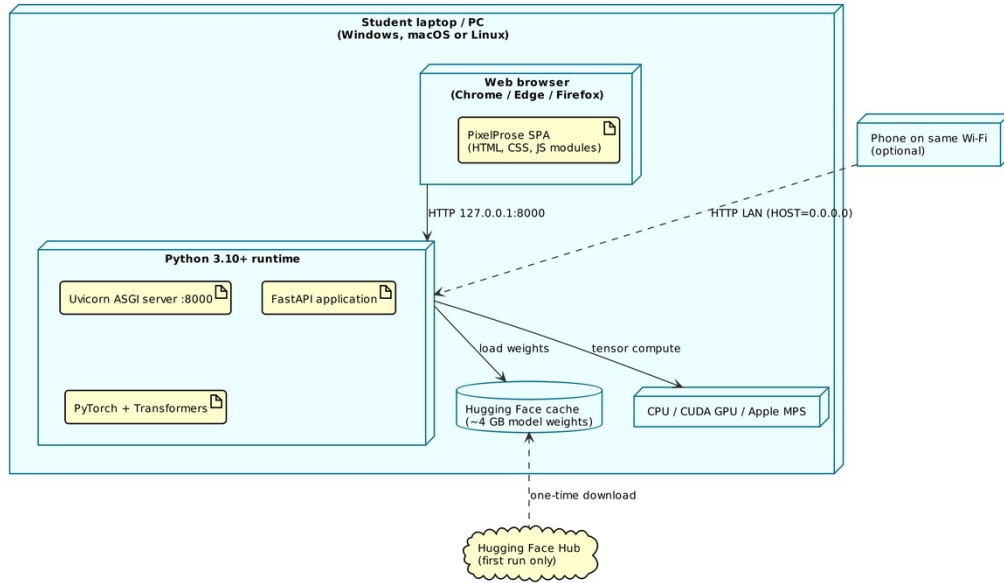
The component diagram in Figure 4.7 shows the modular decomposition. In the browser, feature modules (studio.js, explain.js, chat.js, batch.js, history.js) communicate with the server only through api.js, and history is kept in the browser's localStorage. On the server, the FastAPI routes coordinate the Captioner, Matcher, QuestionAnswerer, image and text utilities and the LRU caches. All neural networks are accessed exclusively through the ModelManager, which isolates the rest of the code from model loading, device selection and concurrency concerns.



**Figure 4.7:** Component diagram

#### 4.2.8 Deployment Diagram

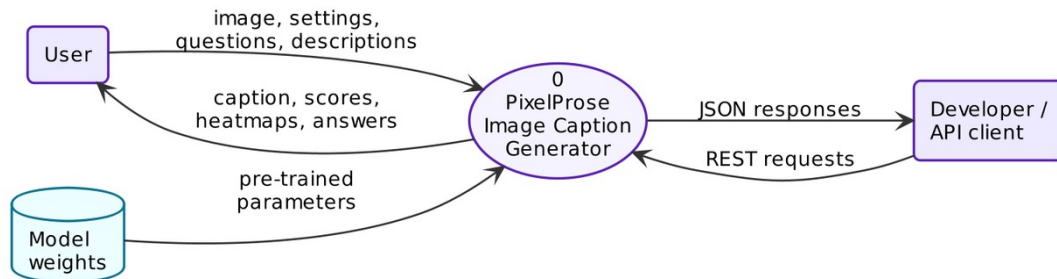
Figure 4.8 shows that the entire system is deployed on a single personal computer. The browser runs the single-page application, and a Python runtime hosts the Uvicorn ASGI server with the FastAPI application and PyTorch. Model weights are read from the local Hugging Face cache and computed on the CPU, a CUDA GPU or Apple MPS. The Hugging Face Hub is contacted only once, to download the weights. Optionally, the server can bind to the local network so that phones on the same Wi-Fi can use the application, which is useful for classroom demonstrations.



**Figure 4.8:** Deployment diagram

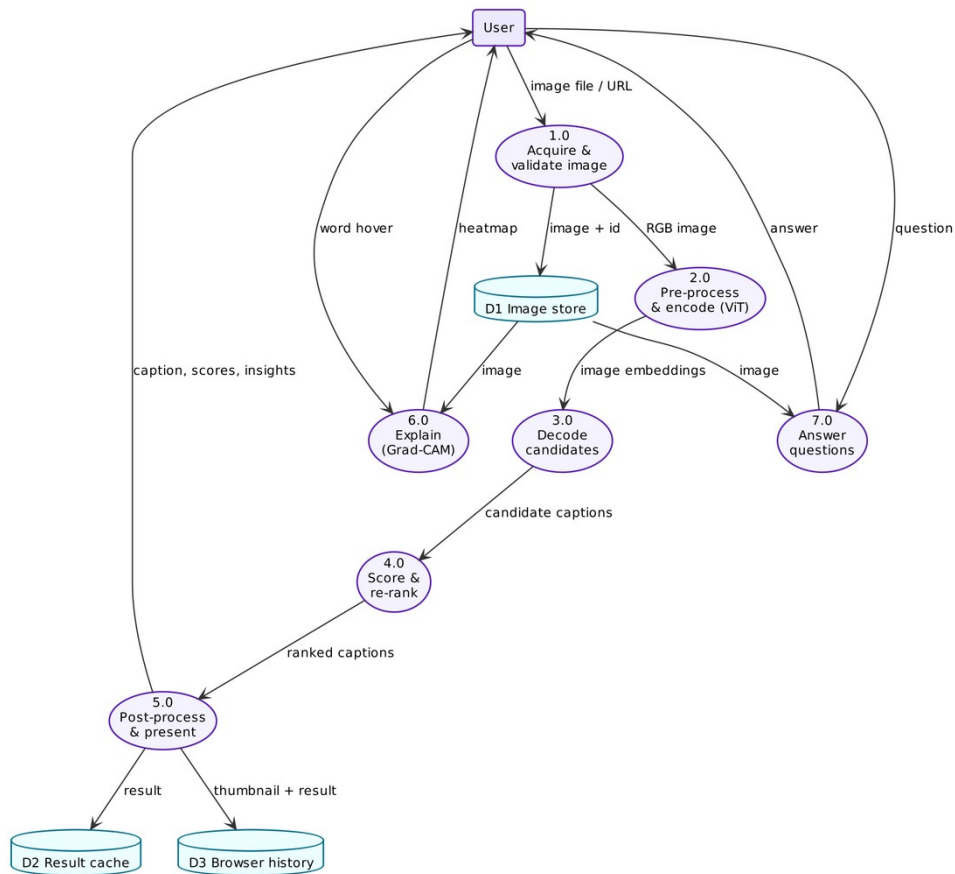
#### 4.2.9 Data Flow Diagrams

The context-level DFD (Level 0) in Figure 4.9 treats PixelProse as a single process. Users send images, settings, questions and descriptions and receive captions, scores, heatmaps and answers; developers exchange REST requests and JSON responses; the model weights supply pre-trained parameters.



**Figure 4.9:** Data flow diagram – Level 0 (context diagram)

The Level-1 DFD in Figure 4.10 decomposes the system into seven processes and three data stores. Process 1.0 acquires and validates the image and stores it in D1 (image store). Processes 2.0 to 5.0 form the captioning pipeline: encoding, decoding candidates, scoring and re-ranking, and post-processing. The final result is written to D2 (result cache) and D3 (browser history). Processes 6.0 (explanation) and 7.0 (question answering) read the stored image from D1, so the user never re-uploads it.



**Figure 4.10:** Data flow diagram – Level 1

### 4.3 System Architecture

PixelProse follows a three-tier, client–server architecture:

- **Presentation tier:** a single-page web application written in plain HTML, CSS and JavaScript ES modules, with no build step. It handles input, rendering, animations, heatmap drawing, speech and history.
- **Application tier:** a FastAPI server exposing REST endpoints. It validates requests with Pydantic, coordinates the services, manages caches and serves the static interface and the OpenAPI documentation.
- **Model tier:** four pre-trained BLIP networks managed by the ModelManager, which loads them lazily (or preloads them in a background thread at start-up), selects the device and precision, and serialises CPU inference to avoid thread contention.

The tiers communicate only through well-defined interfaces: JSON and multipart requests between browser and server, and Python method calls between the routes and the services. This separation allows the interface to be replaced, for example by a

mobile app, or the models to be swapped, for example for BLIP-2, without changing the other tiers.

## 4.4 API Design

The REST API is summarised in Table 4.1. Images are uploaded once; the server returns an *image\_id* (the SHA-1 digest of the file), which follow-up requests reference instead of re-sending the image. Errors use standard HTTP status codes with a human-readable detail message.

**Table 4.1:** REST API endpoints

| Method | Endpoint     | Input                            | Output                                                                                 |
|--------|--------------|----------------------------------|----------------------------------------------------------------------------------------|
| GET    | /api/health  | —                                | Server status, device, per-model state, cache statistics                               |
| GET    | /api/samples | —                                | List of bundled sample images with categories                                          |
| POST   | /api/caption | file / url / image_id + settings | Caption, candidates, confidence, match, word probabilities, palette, metadata, timings |
| POST   | /api/upload  | file                             | image_id and metadata (no captioning)                                                  |
| POST   | /api/explain | image_id, text                   | 24×24 Grad-CAM map per word and overall map                                            |
| POST   | /api/vqa     | image_id, question               | Answer, confidence, latency                                                            |
| POST   | /api/match   | image_id, texts[1..8]            | ITM probability and ITC similarity per text                                            |
| POST   | /api/batch   | files[1..12]                     | Caption per file with scores or error                                                  |

## 4.5 User Interface Design

The interface was designed around four principles: **clarity** (one primary action per screen region), **feedback** (every wait is visualised, every action confirmed), **explainability** (model internals such as candidates, probabilities and attention are first-class elements rather than hidden details) and **delight** (purposeful motion and a cohesive visual identity). The page is organised as a single scrolling document with the following sections:

- **Hero:** product name, tagline, call-to-action buttons, animated statistics and a demo card that loops through real model outputs.
- **Caption Studio:** an input panel (source tabs, preview, settings, Generate button) beside a result panel (caption card, metric rings and five tabs).
- **Batch mode:** multi-file drop zone, progress bar, result grid and export buttons.

- **How it works:** an animated five-step pipeline and six technology cards explaining the AI.
- **Team:** the three project members.
- **Overlays:** history drawer, keyboard-shortcut dialog, toast notifications and tooltips.

The visual language uses a dark glassmorphism theme with violet, indigo and cyan accents, a light theme alternative, the Space Grotesk, Inter and JetBrains Mono typefaces, and design tokens defined once as CSS custom properties. Colour contrast, focus rings and reduced-motion handling were considered from the start. The resulting screens are shown in Chapter 6.

## 4.6 Data Design

PixelProse deliberately avoids a database: images are transient and user data stays in the browser. Three stores exist, as shown in the Level-1 DFD:

- **Image store (server memory):** an LRU cache of the 32 most recent images, keyed by SHA-1 digest, holding the decoded and down-scaled image.
- **Result cache (server memory):** an LRU cache of 64 caption results keyed by (image digest, normalised options). Only deterministic beam-search results are cached.
- **History (browser localStorage):** the last 30 results with a 480-pixel JPEG thumbnail, the full result JSON and a timestamp. When storage is full the oldest entries are dropped automatically.

# CHAPTER 5

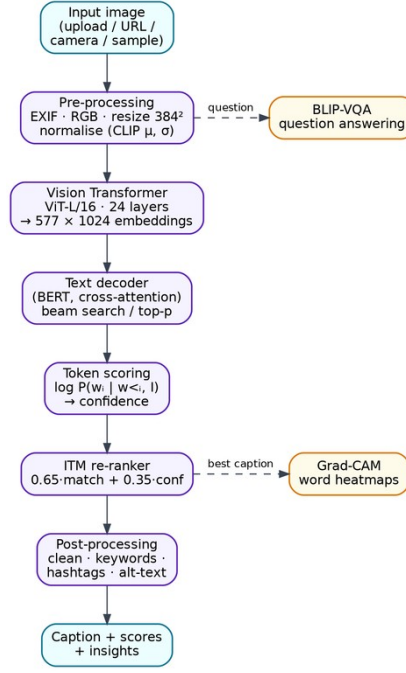
## METHODOLOGY

### 5.1 Introduction

This chapter explains how PixelProse converts pixels into prose. We first present the overall system block diagram and then describe each stage: image pre-processing, the Vision Transformer encoder, the image-grounded text decoder, BLIP pre-training, the decoding algorithms, candidate scoring and re-ranking, Grad-CAM explainability, visual question answering, post-processing and the evaluation metrics. Wherever possible the mathematical formulation is given, followed by the concrete parameter values used in our implementation.

### 5.2 System Block Diagram

Figure 5.1 shows the processing pipeline. An input image is pre-processed and encoded once by the Vision Transformer into 577 embedding vectors. The text decoder, conditioned on these embeddings through cross-attention, generates several candidate captions with beam search (and optionally nucleus sampling). Each candidate is scored by its token probabilities, and the Image–Text Matching (ITM) model independently checks how well it fits the image. The two scores are blended to select the best caption, which is post-processed into a sentence, keywords, hashtags and alt-text. Two side branches reuse the pipeline: Grad-CAM explains the chosen caption word by word, and BLIP-VQA answers user questions.



**Figure 5.1:** System block diagram of PixelProse

### 5.3 Image Pre-processing

Uploaded images arrive in many formats, orientations and sizes. The pre-processing stage makes them uniform:

1. **Validation:** the file must be at most 15 MB, decode successfully with Pillow and be one of JPEG, PNG, WEBP, GIF, BMP or TIFF. A decompression-bomb guard rejects images above 80 megapixels.
2. **Orientation:** the EXIF orientation tag is applied so that photographs taken in portrait mode are upright.
3. **Colour:** images are converted to 3-channel RGB. Transparent PNGs are composited onto white, because otherwise transparent pixels would become black.
4. **Working size:** images larger than 1600 px are down-scaled with Lanczos filtering to save memory, since the models see only 384×384 pixels.
5. **Model input:** the BLIP processor resizes the image to 384×384 with bicubic interpolation, scales pixel values to [0, 1] and normalises each channel with the statistics used during pre-training ((5.1)).

$$\hat{x}_c = (x_c / 255 - \mu_c) / \sigma_c, \quad \mu = (0.4815, 0.4578, 0.4082), \quad \sigma = (0.2686, 0.2613, 0.2758) \quad (5.1)$$

## 5.4 Vision Transformer Encoder

The image encoder is a Vision Transformer [15], ViT-L/16 in BLIP-large. As illustrated in Figure 5.2, the 384×384 image is split into non-overlapping 16×16 patches, giving  $N = (384/16)^2 = 576$  patches. Each patch  $x_p \in \mathbb{R}^{(16 \cdot 16 \cdot 3)}$  is flattened and linearly projected to a D-dimensional embedding ( $D = 1024$ ). A learnable [CLS] token is prepended, and learnable position embeddings  $E_{pos}$  are added so that the model knows where each patch came from:



**Figure 5.2:** The image is resized to 384×384 and split into 24×24 = 576 patches of 16×16 pixels

$$z_0 = [x_{cls}; x_p^1 E; x_p^2 E; \dots; x_p^{576} E] + E_{pos}, \quad E \in \mathbb{R}^{(768 \times 1024)}, \quad E_{pos} \in \mathbb{R}^{(577 \times 1024)} \quad (5.2)$$

The sequence then passes through  $L = 24$  transformer layers. Each layer applies multi-head self-attention (MHSA) and a feed-forward network (MLP), both with pre-layer-normalisation and residual connections:

$$z'_{\ell} = \text{MHSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1} \quad (5.3)$$

$$z_{\ell} = \text{MLP}(\text{LN}(z'_{\ell})) + z'_{\ell}, \quad \ell = 1 \dots L \quad (5.4)$$

Self-attention [13] lets every patch gather information from every other patch. For queries  $Q$ , keys  $K$  and values  $V$  obtained by linear projections of the input:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V \quad (5.5)$$

$$\text{MHSA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(X W_i^Q, X W_i^K, X W_i^V) \quad (5.6)$$

ViT-L/16 uses  $h = 16$  heads of dimension  $d_k = 64$ . The output is a matrix of  $577 \times 1024$  *image embeddings*, a contextualised description of every patch plus a global [CLS] summary, which all subsequent stages consume. In PixelProse this encoding is computed **only once per image** and reused for every candidate caption.

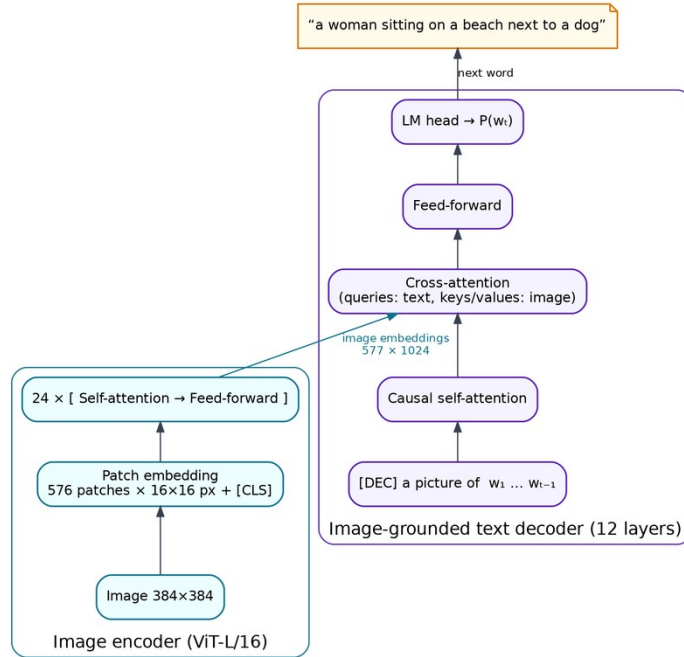
## 5.5 Image-Grounded Text Decoder

The caption is produced by a 12-layer BERT-style [31] transformer decoder (Figure 5.3). Each decoder layer has three sub-layers: *causal* self-attention over the words generated so far (masked so that position  $t$  cannot see positions after  $t$ ), **cross-attention** from the words to the image embeddings, and a feed-forward network. Cross-attention is where vision meets language. The queries come from the text hidden states  $H$  and the keys and values come from the image embeddings  $V$ :

$$\text{CrossAttn}(H, V) = \text{softmax} \left( \frac{(H W^Q)(V W^K)^T}{\sqrt{d_k}} \right) (V W^V) \quad (5.7)$$

A language-modelling head maps the final hidden state to a probability distribution over the 30,524-token WordPiece vocabulary. The probability of a whole caption factorises by the chain rule:

$$P(w_1 \dots w_n | I) = \prod_t P(w_t | w_1 \dots w_{t-1}, I) \quad (5.8)$$



**Figure 5.3:** Architecture of the BLIP captioning model (ViT encoder and image-grounded decoder)

The BLIP captioner was fine-tuned with the textual prompt “a picture of”. PixelProse therefore always starts decoding from the token sequence [DEC] a picture of and strips this prefix from the output. Decoding without it causes BLIP-large to emit nonsense tokens such as “arafed” learned from noisy web alt-text. When the user supplies a *guided prefix* (for example “a painting of”), that prefix replaces the default prompt and is kept in the caption. This is known as conditional captioning.

## 5.6 BLIP Pre-training Objectives

BLIP [19] is pre-trained jointly with three objectives that share the image encoder. Understanding them explains why PixelProse can verify and explain captions without any additional training:

- **Image–Text Contrastive loss (ITC)** aligns the unimodal image and text embeddings. With similarity  $s(I, T) = g_v(v_{cls})^T g_t(t_{cls})$  between projected [CLS] features and a temperature  $\tau$ , the model maximises the softmax probability of the matching pair within a batch ((5.9)).
- **Image–Text Matching loss (ITM)** trains a binary classifier on the multimodal [CLS] output of a text encoder that cross-attends to the image, predicting whether an image–text pair matches. Hard negatives are mined using the contrastive similarity ((5.10)).
- **Language Modelling loss (LM)** trains the image-grounded decoder to generate the caption autoregressively ((5.11)).

$$p_{i2t}(I) = \exp(s(I, T) / \tau) / \sum_m \exp(s(I, T_m) / \tau), \quad L_{itc} = \frac{1}{2} E[ H(y_{i2t}, p_{i2t}) + H(y_{t2i}, p_{t2i}) ] \quad (5.9)$$

$$L_{itm} = E_{(I,T)} [ H(y_{itm}, p_{itm}(I, T)) ] \quad (5.10)$$

$$L_{lm} = - \sum_t \log P_{\theta}(w_t | w_{<t}, I) \quad (5.11)$$

In addition, BLIP's **CapFilt** bootstrapping step uses a fine-tuned captioner to write synthetic captions for 129 million web images and a fine-tuned ITM filter to remove noisy captions, which is why the model generalises well to everyday photographs. PixelProse uses the LM-trained decoder to *generate*, the ITM head to *verify* and *explain*, and the ITC projections to report a fast similarity score.

## 5.7 Decoding Strategies

Generating a caption means searching for a high-probability word sequence. Exhaustive search is impossible because the number of sequences grows as  $|V|^n$ . PixelProse implements three strategies, exposed to users as caption styles.

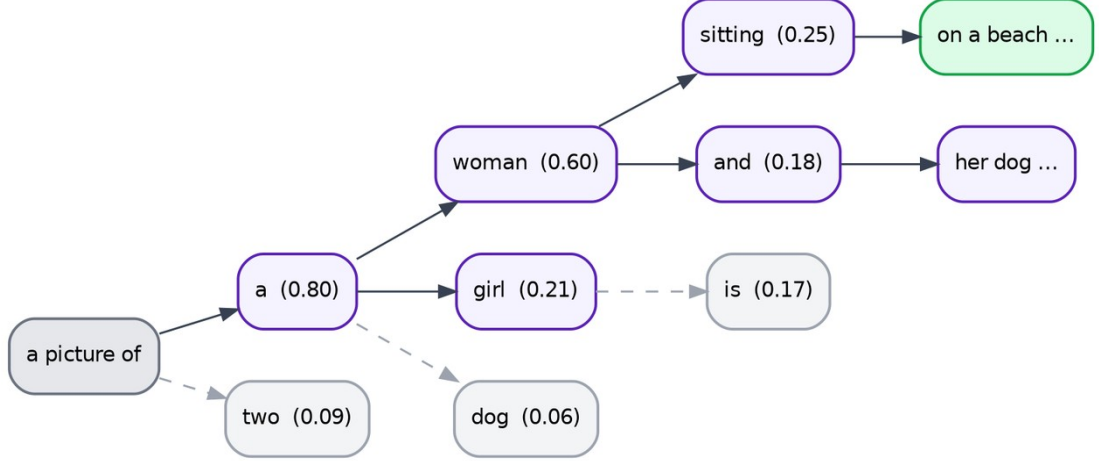
### 5.7.1 Greedy Decoding (baseline)

Greedy decoding picks the single most probable word at each step,  $w_t = \arg\max_w P(w | w_{<t}, I)$ . It is fast but short-sighted: an early locally-best word can lead to a poor sentence. It is used only as the baseline in our evaluation.

### 5.7.2 Beam Search (Balanced mode)

Beam search keeps the  $k$  most probable partial sentences (the *beam*) at each step, as illustrated in Figure 5.4. Candidates are ranked by their length-normalised log-probability:

$$\text{score}(y) = (1 / |y|^\alpha) \cdot \sum_t \log P(y_t | y_{<t}, I), \quad \alpha = 1 \quad (5.12)$$



**Figure 5.4:** Beam search keeps the  $k$  best partial captions at every step (pruned paths shown dashed)

#### Algorithm – Beam search with candidates:

1. Initialise the beam with the prompt sequence [DEC] a picture of and score 0.
2. For each step  $t = 1 \dots \text{max\_new\_tokens}(30)$ : expand every hypothesis in the beam with every vocabulary token and compute the new cumulative log-probability.
3. Apply the repetition penalty ((5.13)) and block any token that would repeat an already generated 3-gram.
4. Keep the  $k = 5$  highest-scoring hypotheses; move hypotheses that produced the end token [SEP] (after at least 5 new tokens) to the finished list.
5. Stop when  $k$  hypotheses are finished and no open hypothesis can beat them (early stopping).
6. Return the top  $\text{num\_candidates}$  finished hypotheses as candidate captions.

$$z_w \leftarrow z_w / \theta \text{ if } z_w > 0, \text{ else } z_w \cdot \theta, \text{ for every previously generated token } w, \quad \theta = 1.1 \quad (5.13)$$

### 5.7.3 Nucleus (Top-p) Sampling (Detailed and Creative modes)

Beam search tends to produce safe but generic sentences [28]. Nucleus sampling instead samples each word from the smallest set of words  $V^{(p)}$  whose cumulative probability exceeds  $p$ , after sharpening or flattening the distribution with a temperature  $T$ :

$$P\_T(w) = \exp(z\_w / T) / \sum_v \exp(z\_v / T) \quad (5.14)$$

$$V^{(p)} = \text{smallest set with } \sum_{(w \in V^{(p)})} P\_T(w) \geq p, \quad P'(w) = P\_T(w) / \sum_{(v \in V^{(p)})} P\_T(v) \text{ for } w \in V^{(p)}, \text{ else } 0 \quad (5.15)$$

PixelProse uses  $p = 0.9$  and a default temperature  $T = 0.9$  (adjustable from 0.3 to 1.5 through the *Creativity* slider). The *Detailed* mode merges  $\lceil n/2 \rceil + 1$  beam candidates with  $\lceil n/2 \rceil + 1$  sampled candidates, and the *Creative* mode uses sampling only.

## 5.8 Candidate Scoring and Re-ranking

### 5.8.1 Language-Model Confidence

After decoding, duplicate candidates (after cleaning) are removed and all remaining candidates are scored in **one batched teacher-forced forward pass** of the decoder over the shared image embeddings. For a candidate with generated tokens  $w_1 \dots w_n$  (including the end token), its confidence is the geometric mean of the token probabilities, which equals the inverse of its perplexity:

$$\text{conf} = \exp\left(\frac{1}{n} \sum_t \log P(w_t | w_{<t}, I)\right) = 1 / \text{PPL} \quad (5.16)$$

WordPiece tokens are merged into words for display (for example `sky + ##scraper` → `skyscraper`), and a word's probability is the product of its pieces' probabilities. These per-word probabilities are drawn as the *word-by-word confidence* bar chart in the interface.

### 5.8.2 Image–Text Matching Verification

Language-model confidence measures fluency, not truthfulness: a fluent sentence may mention an object that is not in the picture. PixelProse therefore re-reads each candidate with the separately fine-tuned **BLIP-ITM** model. The candidate text is encoded by a BERT encoder that cross-attends to the image, and the ITM head outputs the probability that the pair matches:

$$P\_itm(I, T) = \text{softmax}(W\_itm \cdot h\_cls(I, T))_1 \quad (5.17)$$

The final score blends grounding and fluency, and the highest-scoring candidate becomes the caption:

$$S(T) = 0.65 \cdot P\_itm(I, T) + 0.35 \cdot \text{conf}(T) \quad (5.18)$$

The weights favour grounding while still penalising ungrammatical candidates. The contrastive cosine similarity  $\cos(g\_v(v\_cls), g\_t(t\_cls))$  from the ITC projections is also

reported. It is cheaper but less discriminative than ITM, and it powers the *similarity* value in the Match tab.

## 5.9 Explainability with Grad-CAM

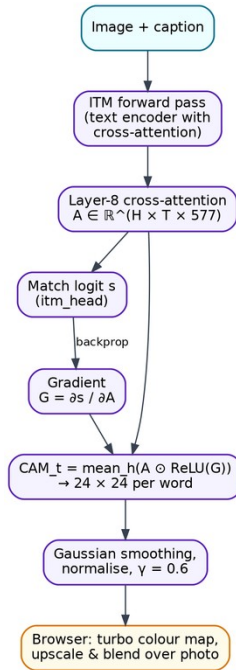
To show *where* the model looked for each word, we apply Grad-CAM [22] to the cross-attention of the ITM model, following the grounding approach of ALBEF and BLIP [18] (Figure 5.5). Let  $A \in \mathbb{R}^{(H \times T \times 577)}$  be the cross-attention probabilities of layer 8 ( $H = 12$  heads,  $T$  text tokens,  $1 + 576$  image tokens) and  $s$  the ITM *match* logit. We back-propagate  $s$  to the attention map and weight the attention by the positive part of its gradient:

$$G = \partial s / \partial A \quad (5.19)$$

$$\text{CAM}_t = (1/H) \sum_h A_{(h,t,1:)} \odot \max(0, G_{(h,t,1:)} ) \in \mathbb{R}^{(24 \times 24)} \quad (5.20)$$

The positive gradient keeps only the attention that *increased* the match score, which makes the maps word-specific rather than generic saliency. Raw maps are peaky, often only two or three patches, so each map is smoothed with a  $5 \times 5$  Gaussian kernel ( $\sigma = 1$  patch), min-max normalised and gamma-corrected ((5.21)) to reveal weaker but relevant regions. Word pieces are averaged into words, stop-words are flagged, and an overall map is computed as the mean over content words. The browser colours the  $24 \times 24$  grid with a turbo colour map, upsamples it with smoothing, and blends it over the photograph at a user-selected opacity.

$$\tilde{M} = ( (M - \min M) / (\max M - \min M) )^\gamma, \quad \gamma = 0.6 \quad (5.21)$$



**Figure 5.5:** Grad-CAM pipeline for word-level explanations

## 5.10 Visual Question Answering

For the Ask feature PixelProse uses **BLIP-VQA** (ViT-B/16 backbone). The image is encoded by the ViT. The question, for example “what is the woman wearing?”, is encoded by a BERT encoder that cross-attends to the image embeddings, producing a question representation grounded in the picture. An answer decoder then generates the answer autoregressively with beam search (3 beams, at most 12 tokens) while cross-attending to the question representation. The answer confidence is exp of the length-normalised beam score. Questions are normalised (whitespace collapsed, a question mark appended, length limited to 200 characters) before encoding.

## 5.11 Post-processing

The raw decoder output is converted into user-facing text by a deterministic post-processing algorithm:

1. Remove the conditioning prompt (“a picture of”) if it was not supplied by the user.
2. Delete known BLIP artefact tokens (“arafed”, “araffe”, ...) and a leading filler phrase (“there is”, “there are”, “this is”).
3. Collapse immediate word repetitions (“the the” → “the”) and patterns such as “lava and lava” → “lava”.

4. Normalise whitespace and punctuation; discard candidates with fewer than two words.
5. Produce the display sentence (capitalised, full stop), keywords (content words after stop-word removal), hashtags, an accessible HTML alt-text snippet and the word count.

In parallel, the image analysis module extracts the six dominant colours by median-cut quantisation of a 160-pixel thumbnail, together with the resolution, aspect ratio, orientation, format, file size and the mean brightness of the image, which is classified as bright, balanced or dark.

## 5.12 Evaluation Metrics

To measure caption quality objectively we implemented the three most widely used reference-based metrics, following their original definitions and the MS-COCO evaluation toolkit. Let  $c$  be a candidate caption and  $S = \{s_1, \dots, s_m\}$  its human references.

### 5.12.1 BLEU

BLEU [24] computes the *modified n-gram precision*  $p_n$ , the fraction of candidate  $n$ -grams that also occur in some reference with counts clipped to the maximum reference count, combines  $n = 1 \dots 4$  with a geometric mean and multiplies by a brevity penalty BP that punishes captions shorter than the references ( $c$ : candidate length,  $r$ : effective reference length, summed over the corpus):

$$\text{BLEU-N} = \text{BP} \cdot \exp\left(\sum_{n=1}^N (1/N) \log p_n\right), \quad \text{BP} = 1 \text{ if } c > r, \text{ else } \exp(1 - r/c) \quad (5.22)$$

### 5.12.2 ROUGE-L

ROUGE-L [25] uses the length of the longest common subsequence (LCS) between candidate and reference, which rewards in-order matches without requiring them to be consecutive. Precision and recall use the best reference, and they are combined into an F-measure with  $\beta = 1.2$ :

$$R = \max_j \text{LCS}(c, s_j) / |s_j|, \quad P = \max_j \text{LCS}(c, s_j) / |c|, \quad \text{ROUGE-L} = (1 + \beta^2) R P / (R + \beta^2 P) \quad (5.23)$$

### 5.12.3 CIDEr-D

CIDEr [26] measures consensus with the references using TF-IDF-weighted  $n$ -gram vectors, so that informative words (“giraffe”) count more than frequent ones (“a”,

“the”). For  $n$ -gram  $\omega_k$  with count  $h_k(s)$  in sentence  $s$  and document frequency over the  $|I|$  images of the test set, the weight is  $g_k(s) = h_k(s) \cdot \log(|I| / df(\omega_k))$ . The CIDEr-D variant clips candidate weights and adds a Gaussian length penalty to prevent gaming by repetition or length:

$$\text{CIDEr-D}_n(c, S) = (10/m) \sum_j \exp(-(l_c - l_{s_j})^2 / 2\sigma^2) \cdot (\min(g^n(c), g^n(s_j)) \cdot g^n(s_j)) / (\|g^n(c)\| \|g^n(s_j)\|), \quad \sigma = 6 \quad (5.24)$$

$$\text{CIDEr-D}(c, S) = (1/4) \sum_{n=1}^4 \text{CIDEr-D}_n(c, S) \quad (5.25)$$

In addition we report the mean ITM match probability of the chosen captions (a reference-free grounding measure) and the latency per image.

## 5.13 Overall Algorithm

The complete caption-generation procedure, as implemented in `app/services/captioner.py`, is summarised below.

```

Algorithm PixelProse-Caption(image I, options opt)
1  I ← validate, EXIF-rotate, convert to RGB, downscale(I)
2  key ← (SHA1(I), normalise(opt)); if opt.strategy = beam and key ∈ Cache: return
   Cache[key]
3  x ← resize(I, 384×384); normalise with CLIP mean/std           ▷ Eq. norm
4  V ← ViT(x)                                                    ▷ 577 × 1024, computed once
5  prompt ← opt.prompt or "a picture of"
6  C ← ∅
7  if strategy ∈ {beam, hybrid}:    C ← C ∪ BeamSearch(V, prompt, k = 5)
8  if strategy ∈ {hybrid, sampling}: C ← C ∪ NucleusSample(V, prompt, p = 0.9, T)
9  C ← unique( clean(c) for c in C )
10 for each c in C: conf(c) ← exp(mean log P(c | V))           ▷ one batched pass
11 if opt.rerank:    for each c in C: S(c) ← 0.65·P_itm(I, c) + 0.35·conf(c)
12 else:            S(c) ← conf(c)
13 best ← argmax_c S(c)
14 result ← {best, sorted C, word probabilities, keywords, hashtags, palette,
   timings}
15 Cache[key] ← result; return result

```

*Listing: Pseudocode of the caption-generation algorithm*

The time complexity is dominated by the single ViT forward pass,  $O(L \cdot N^2 \cdot D)$  for  $N = 577$  tokens, and by beam decoding,  $O(k \cdot n)$  decoder steps each costing  $O(n \cdot D + N \cdot D)$  due to self- and cross-attention. Encoding the image once instead of once per candidate, and scoring all candidates in one batched pass, are the two key optimisations of our pipeline.

# CHAPTER 6

## IMPLEMENTATION DETAILS

### 6.1 Introduction

This chapter describes how the design of Chapter 4 and the methodology of Chapter 5 were implemented. We present the development environment and technology stack, the source-code organisation, the key backend and frontend modules with representative code, screenshots of the working system with a description of each, the testing performed and the deployment procedure. The complete source code is maintained in a Git repository.

### 6.2 Development Environment and Technology Stack

Table 6.1: Technology stack

| Layer         | Technology                                                   | Role in PixelProse                                                                            |
|---------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| AI framework  | PyTorch 2.x [32]                                             | Tensor operations, inference, autograd for Grad-CAM                                           |
| Model library | Hugging Face Transformers 4.5x [33]                          | BlipForConditionalGeneration, BlipForImageTextRetrieval, BlipForQuestionAnswering, processors |
| Models        | Salesforce BLIP [19]                                         | Captioning (large, base), ITM, VQA                                                            |
| Web API       | FastAPI [34] + Uvicorn                                       | REST endpoints, validation (Pydantic), OpenAPI documentation                                  |
| Imaging       | Pillow, NumPy                                                | Decoding, EXIF, resizing, palette extraction, statistics                                      |
| Frontend      | HTML5, CSS3, JavaScript ES modules                           | Single-page interface without a build step                                                    |
| Browser APIs  | Canvas 2D, Web Speech, MediaDevices, Clipboard, localStorage | Heatmaps and cards, text-to-speech, webcam, paste, history                                    |
| Testing       | pytest, FastAPI TestClient                                   | Unit and API tests with mocked models                                                         |
| Tooling       | Git/GitHub, VS Code, PlantUML, Matplotlib, Playwright        | Version control, editing, diagrams, charts, UI screenshots                                    |

### 6.3 Project Structure

The repository separates concerns into clear top-level folders: **app/** contains the FastAPI application (config.py for settings, main.py for routes, schemas.py for request models) and the **services/** package with one module per responsibility (model\_manager, captioner, matcher, vqa, image\_utils, text\_utils, metrics and store). **static/** holds the single-page interface (index.html, one stylesheet and twelve JavaScript

modules). **scripts/** contains the model download, offline conversion and evaluation tools, **sample\_images/** the 44 test images with their reference captions, **tests/** the pytest suite, and **docs/**, **presentation/** and **report/** the documentation, slide deck and this report, each generated from source. `run.py` starts the web server and `cli.py` provides the command-line interface.

## 6.4 Backend Implementation

### 6.4.1 Configuration and Model Management

All settings (model sources, device, feature toggles, limits) are read once from environment variables or a `.env` file into an immutable `Settings` object, so the same code can run with Hugging Face Hub models or locally converted weights. The **ModelManager** loads each model at most once using double-checked locking, selects CUDA, Apple MPS or CPU automatically, and uses half precision on GPUs. During testing we found that PyTorch's worker threads spin-wait: when the browser's rendering occupied one CPU core, a caption that normally takes about 5 seconds took more than 90 seconds. The manager therefore reserves one core and serialises CPU inference with a shared re-entrant lock:

Models are preloaded in a background thread when the server starts, so the web page is usable immediately while a status pill reports progress through `/api/health`.

### 6.4.2 Caption Generation

The **Captioner** implements the pipeline of Section 5.13. The key optimisation is that the Vision Transformer runs once and every decoding call reuses the image embeddings. It achieves this by calling the text decoder directly with the same preparation that BLIP's `generate()` performs internally:

```
def _encode(self, lm, image, prompt):
    inputs = lm.processor(images=image, text=prompt, return_tensors="pt")
    pixel_values = inputs["pixel_values"].to(device, dtype)
    image_embeds = lm.model.vision_model(pixel_values=pixel_values)[0] # once per
image
    input_ids = inputs["input_ids"].to(device)
    input_ids[:, 0] = lm.model.config.text_config.bos_token_id # [CLS] ->
[DEC]
    return image_embeds, input_ids[:, :-1], inputs["attention_mask"].to(device)[: , :-1]
```

*Listing: Encoding the image once (captioner.py)*

Candidates from beam search and sampling are cleaned and de-duplicated, padded into one batch and scored with a single teacher-forced pass that gathers the log-probability of every generated token. The ITM Matcher then scores all candidates in one batch, and the blended score of Eq. (5.18) selects the caption. Every stage is timed, and the timings are returned for the *Pipeline timing* panel of the interface.

### 6.4.3 Matching and Grad-CAM Grounding

The **Matcher** loads BLIP-ITM with the eager attention implementation so that attention probabilities are materialised and differentiable. Grounding runs a forward pass with `output_ attentions=True`, takes the layer-8 cross-attention and back-propagates the match logit to it with `torch.autograd.grad`, which computes only the gradient needed without updating any weights:

```
with lm.lock, torch.enable_grad():
    out = model.text_encoder(input_ids=ids, attention_mask=mask,
                             encoder_hidden_states=image_embeds,
                             encoder_attention_mask=image_mask,
                             output_ attentions=True)
    cross = out.cross_ attentions[8] # (1, heads, tokens, 1 +
576)
    logit = model.itm_head(out.last_hidden_state[:, 0, :])[:, 1].sum()
    grads = torch.autograd.grad(logit, cross)[0]
    cam = (cross.detach() * grads.clamp(min=0)).mean(dim=1)[0, :, 1:] # Eq. (5.20)
    cam = _smooth(cam.reshape(-1, 24, 24)) # Gaussian 5x5
```

*Listing: Grad-CAM on cross-attention (matcher.py)*

### 6.4.4 REST API, Validation and Caching

FastAPI routes are thin: they resolve the image (file, URL or image\_id), build a CaptionOptions object, consult the cache and delegate to the services. Request bodies are validated by Pydantic models with length and range limits. A URL image is downloaded by the server only after its host resolves to a public IP address, and the check is repeated after redirects to prevent server-side request forgery. Custom exception handlers convert image errors to HTTP 400 and model-loading failures to HTTP 503 with guidance for offline setup. JSON responses larger than 1 KB are gzip-compressed. The Swagger UI documentation is served from local files, so it also works without internet.

### 6.4.5 Offline Model Conversion

Some campus networks block huggingface.co. The script `convert_original_blip.py` downloads Salesforce's original research checkpoints from Google Cloud Storage,

renames their parameters to the Transformers layout, interpolates position embeddings where needed (the VQA checkpoint was trained at 480 px) and writes ready-to-use model folders. While developing it we discovered that BlipTextConfig defaults to 8 attention heads whereas BLIP uses 12. Because the weight shapes are identical, the wrong configuration loads without any error, yet the ITM probability of a correct caption dropped from 0.996 to 0.02. Validating against a known reference output exposed this silent bug.

## 6.5 Frontend Implementation

The interface is a single HTML page styled by one CSS file with design tokens (colours, radii, easing curves) defined as custom properties for dark and light themes. Behaviour is split into twelve ES modules, listed in Table 6.2.

**Table 6.2:** Frontend JavaScript modules

| Module     | Responsibility                                                                           |
|------------|------------------------------------------------------------------------------------------|
| main.js    | Bootstraps all modules, theme handling, keyboard shortcuts                               |
| api.js     | Fetch wrapper with readable error messages and status codes                              |
| ui.js      | Shared helpers: toasts, tooltips, tabs, sliders, ripples, sparkles, count-ups, storage   |
| effects.js | Scroll reveals, particle network canvas, word rotator, magnetic buttons, tilt, demo card |
| studio.js  | Input sources, settings, generation flow, loading animation, result rendering            |
| explain.js | Heatmap rendering on a canvas overlay, word hover, play mode, opacity                    |
| chat.js    | Ask (VQA chat) and Match (description scoring) tabs                                      |
| batch.js   | Sequential batch captioning with progress and CSV/JSON export                            |
| history.js | localStorage history drawer with restore, export and clear                               |
| camera.js  | Webcam start, stop, flip and capture                                                     |
| card.js    | 1080×1350 shareable caption card drawn on a canvas                                       |
| status.js  | Polls /api/health and updates the model status pill                                      |

Heatmaps are rendered efficiently: the 24×24 grid is written into a tiny 24×24 bitmap coloured with a turbo colour map (with transparency increasing for weak values) and then drawn onto a canvas that exactly overlays the displayed image, letting the browser upscale it smoothly with a blur filter. Decorative animations are paused while inference runs and animation loops stop when their elements are off-screen, which keeps CPU free for the model. The interface honours prefers-reduced-motion and provides keyboard shortcuts (Ctrl+Enter generate, R regenerate, C copy, S speak, H history, T theme, ? help, Esc close).

## 6.6 System Implementation Screenshots

This section presents the working system. All screenshots were captured from the running application.

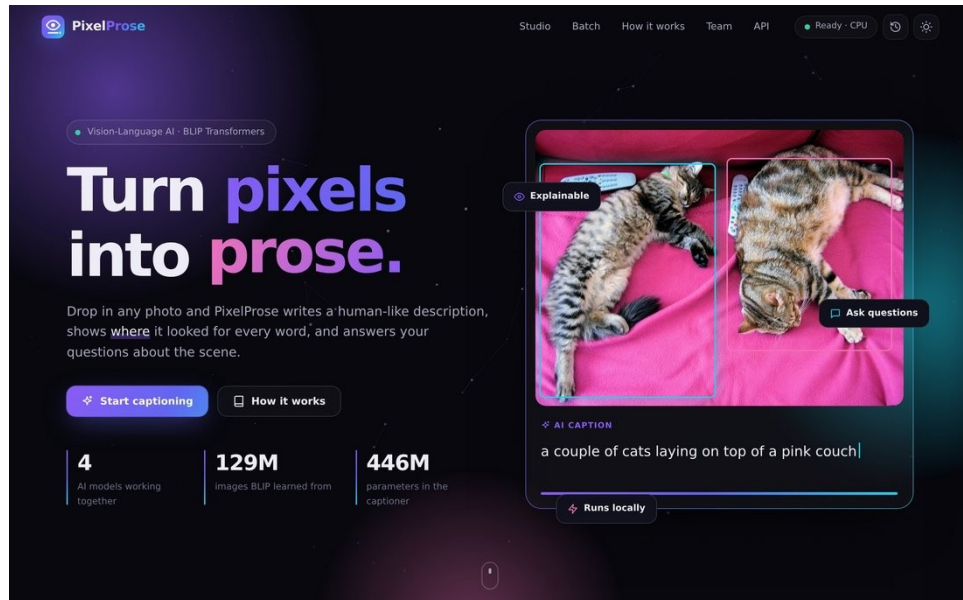


Figure 6.1: Home page (hero section) of PixelProse

Figure 6.1 shows the landing section. The animated headline rotates through words (“prose”, “stories”, “captions”), a particle network reacts to the mouse, and a demo card on the right loops through real model outputs with a scanning animation and typewriter text. The navigation bar contains a live model-status pill (“Ready · CPU”), the history button and the theme toggle.

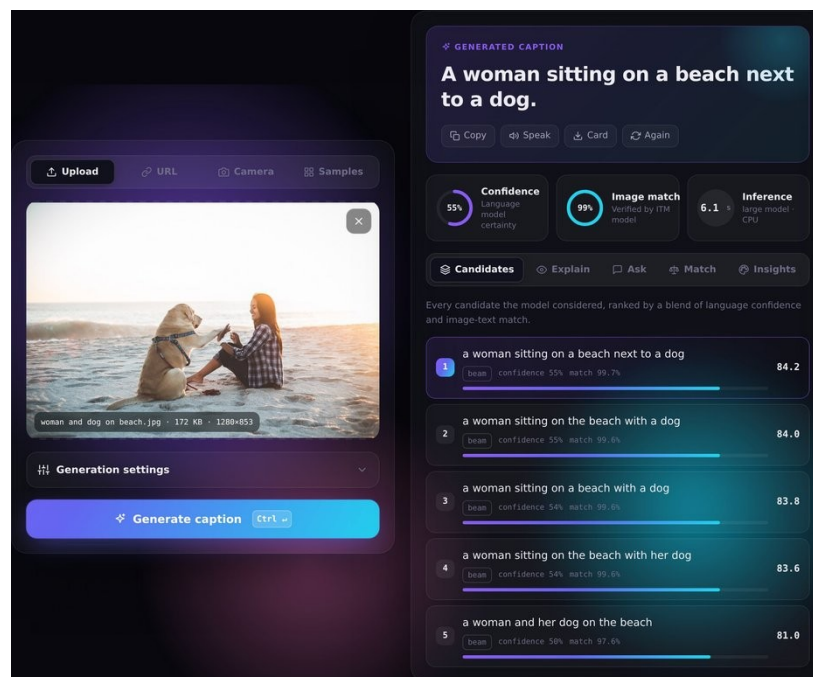
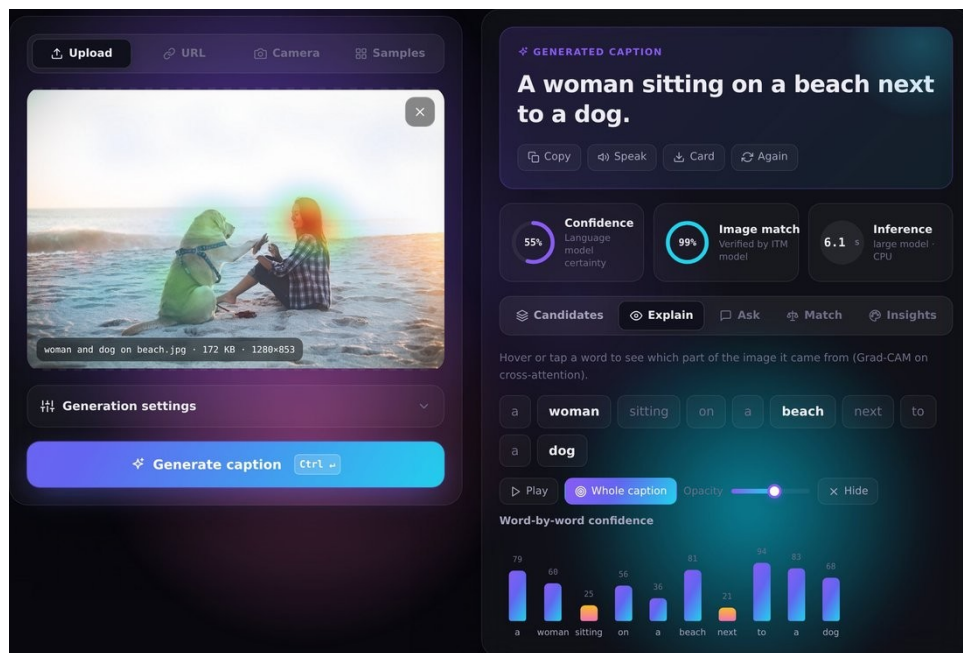


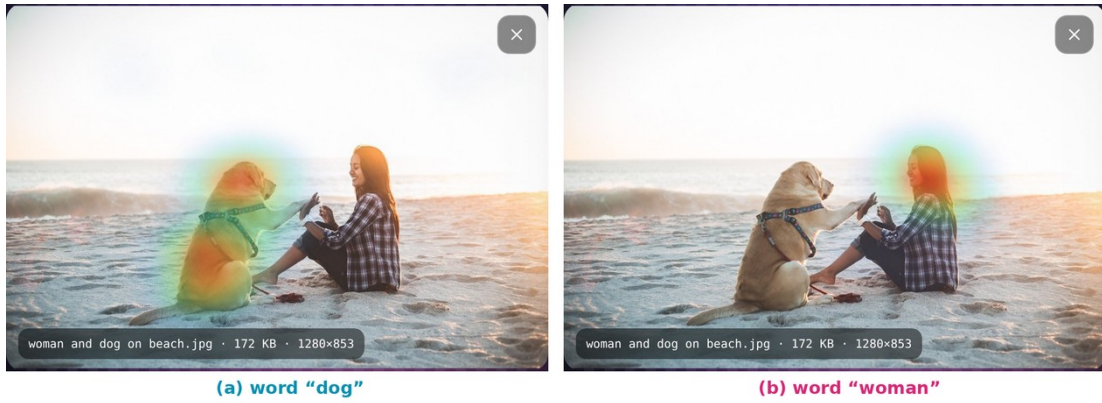
Figure 6.2: Caption Studio with a generated caption and ranked candidates

While a caption is being generated, a scanning beam sweeps the image over a 24×24 grid that matches the patch grid of the Vision Transformer, and the result panel shows an animated neural network with a live checklist of the pipeline stages. Figure 6.2 shows the result for a sample image. The caption “A woman sitting on a beach next to a dog.” is revealed word by word in the caption card, with Copy, Speak, Card and Again actions. Three metric cards show the language-model confidence (55 %), the ITM image-match (99 %) and the inference time. The Candidates tab lists all five beam candidates with their source, confidence, match and blended score. Clicking a candidate promotes it to be the caption.



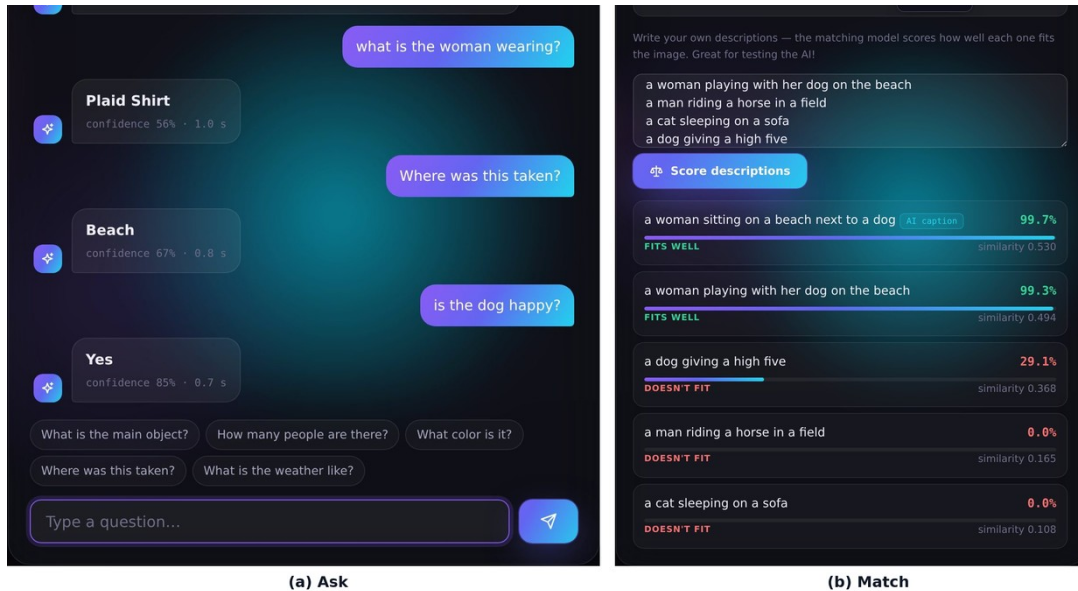
**Figure 6.3:** Explain tab: attention heatmap and word-by-word confidence

Figure 6.3 shows the Explain tab. Each caption word is a button, and hovering over it overlays that word's Grad-CAM heatmap on the image. The *Whole caption* button shows the combined map, *Play* animates through the content words and a slider sets the opacity. Below, the word-by-word confidence chart shows the probability of every word; words below 25 % (here “sitting” and “next”) are highlighted in orange as points of hesitation.



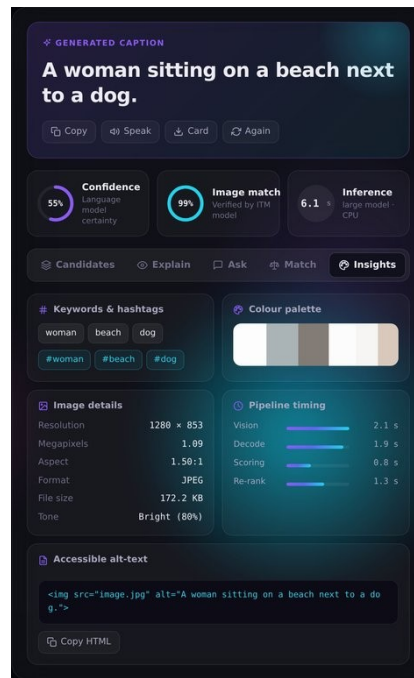
**Figure 6.4:** Word-level heatmaps produced by Grad-CAM

Figure 6.4 compares two words of the same caption. For “dog” the heat covers the dog’s head and body; for “woman” it moves to the woman’s face and upper body. This demonstrates that the model grounds each word in the corresponding region of the image.



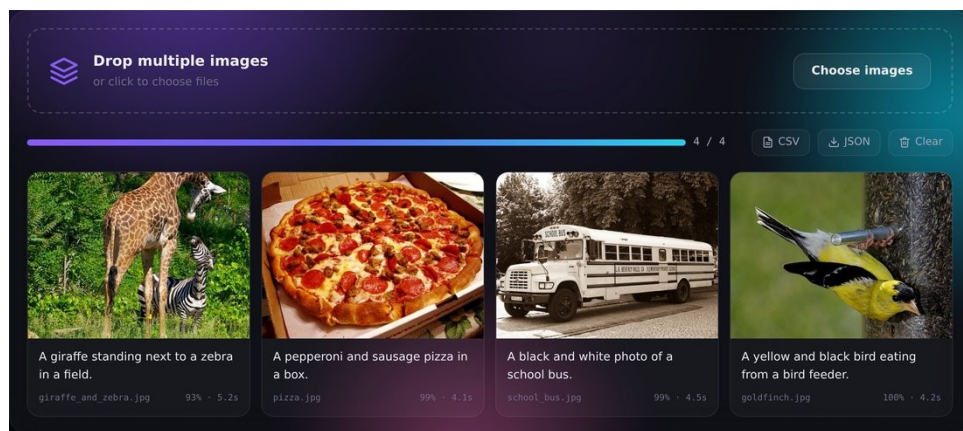
**Figure 6.5:** Ask tab (visual question answering) and Match tab (scoring user descriptions)

Figure 6.5(a) shows the question-answering chat. Questions can be typed or chosen from suggestion chips, and each answer shows its confidence and latency: “what is the woman wearing?” → *plaid shirt*, “where was this taken?” → *beach*, “is the dog happy?” → *yes*. Figure 6.5(b) shows the Match tab, where users type their own descriptions. The AI caption (99.7 %) and “a woman playing with her dog on the beach” (99.3 %) *fit well*. “A dog giving a high five” receives 29.1 %, since it is partly right but misses the woman, while “a man riding a horse in a field” and “a cat sleeping on a sofa” receive 0.0 %.



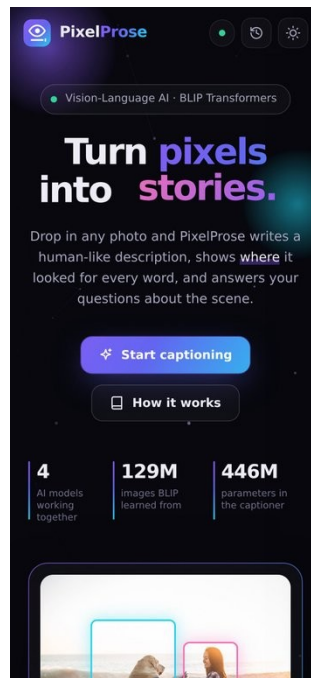
**Figure 6.6:** Insights tab: keywords, palette, metadata, timing and alt-text

Figure 6.6 shows the Insights tab with keywords and hashtags, the dominant colour palette (click to copy hex codes), image details, the time taken by each pipeline stage and an accessible HTML alt-text snippet ready to paste into a web page.



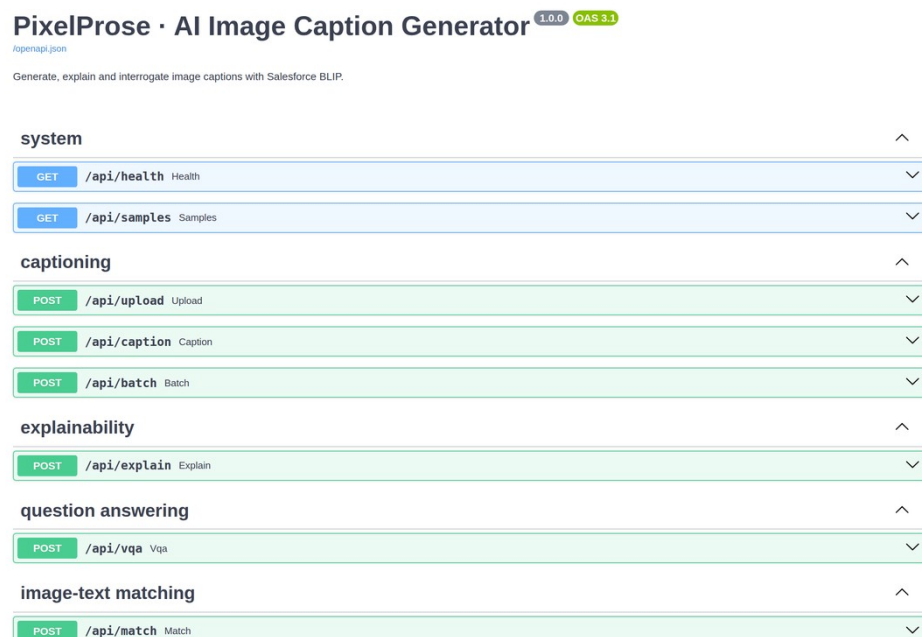
**Figure 6.7:** Batch mode with export to CSV and JSON

Figure 6.7 shows batch mode: several images are dropped at once, captioned one after another with individual scanning animations and a progress bar, and the results can be exported as CSV or JSON.



**Figure 6.8:** Responsive layout on a mobile phone

Figure 6.8 shows the responsive layout on a phone. A light theme is also available; it follows the operating-system preference by default and is remembered per browser.



**Figure 6.9:** Interactive REST API documentation (Swagger UI)

Figure 6.9 shows the automatically generated, interactive API documentation, where every endpoint can be tried directly from the browser.

```
Terminal — PixelProse CLI

$ python cli.py sample_images/animals/giraffe_and_zebra.jpg
  sample_images/people/street_with_bus.jpg --ask "what animals are there?" --ask
  "what color is the bus?"
Loading models on cpu (first run downloads them)...

sample_images/animals/giraffe_and_zebra.jpg
A giraffe standing next to a zebra in a field.
confidence 66.5% · match 92.9% · 4.4s · #giraffe #zebra #field
2. a giraffe and a zebra in a grassy field (0.822)
3. a giraffe and a zebra standing in a field (0.814)
Q: what animals are there? giraffe and zebra
Q: what color is the bus? no bus

sample_images/people/street_with_bus.jpg
A group of people crossing a street in front of a bus.
confidence 58.5% · match 61.8% · 3.6s · #group #people #crossing #street #bus
2. a group of people crossing the street in front of a bus (0.561)
3. a group of people walking across a street next to a bus (0.558)
Q: what animals are there? none
Q: what color is the bus? blue
```

**Figure 6.10:** Command-line interface output

Figure 6.10 shows the command-line tool captioning two images and answering two questions about each. It uses the same services as the web application and can export CSV or JSON.

## 6.7 Testing

Testing was carried out at three levels. **Unit tests** check text processing (prompt stripping, artefact removal, repetition collapse, keywords, hashtags), image handling (format validation, transparency flattening, down-scaling, palette, metadata, SSRF blocking of private URLs) and the metric implementations (a perfect caption scores 1.0 on BLEU and ROUGE-L; a wrong caption scores lower on CIDEr-D). **API tests** exercise every endpoint through FastAPI's TestClient with the neural networks replaced by fast fakes, so the suite of 36 tests runs in about 8 seconds. **End-to-end tests** drove the real interface in a headless Chromium browser with Playwright, covering sample selection, generation, every tab, batch mode, theme switching and the phone layout. They also detected and led to the fix of a horizontal-overflow bug on phones. Table 6.3 lists representative test cases.

**Table 6.3:** Representative test cases

| ID    | Test case                           | Expected result                                  | Req.       | Status |
|-------|-------------------------------------|--------------------------------------------------|------------|--------|
| TC-01 | Upload a valid JPEG and generate    | Caption, candidates, scores and palette returned | FR-1, 3, 4 | Pass   |
| TC-02 | Upload a text file renamed as image | HTTP 400 “not a readable image”                  | FR-2       | Pass   |
| TC-03 | Upload a 16 MB image                | Rejected with size message                       | FR-2       | Pass   |

| ID    | Test case                          | Expected result                           | Req.  | Status |
|-------|------------------------------------|-------------------------------------------|-------|--------|
| TC-04 | URL pointing to 127.0.0.1          | Rejected as private host (SSRF)           | FR-1  | Pass   |
| TC-05 | Generate twice with beam search    | Second response served from cache         | FR-3  | Pass   |
| TC-06 | Guided prefix “a painting of”      | Caption starts with the prefix            | FR-10 | Pass   |
| TC-07 | Invalid parameter (99 candidates)  | HTTP 422 validation error                 | FR-10 | Pass   |
| TC-08 | Explain a caption                  | 24×24 map per word + overall map          | FR-7  | Pass   |
| TC-09 | Ask “how many cats are there?”     | Answer “2”                                | FR-8  | Pass   |
| TC-10 | Match correct vs wrong description | Correct $\gg$ wrong (99.7 % vs 0.0 %)     | FR-9  | Pass   |
| TC-11 | VQA with unknown image_id          | HTTP 404 “upload again”                   | FR-8  | Pass   |
| TC-12 | Batch with 2 images + 1 text file  | 2 captions, 1 error entry                 | FR-13 | Pass   |
| TC-13 | Restore an entry from history      | Result shown; Explain/Ask work again      | FR-14 | Pass   |
| TC-14 | Open /docs offline                 | Swagger UI loads from local files         | FR-15 | Pass   |
| TC-15 | Phone viewport 390 px              | No horizontal scroll; all features usable | NFR   | Pass   |
| TC-16 | Transparent PNG input              | Flattened on white, not black             | FR-2  | Pass   |

## 6.8 Deployment

PixelProse is distributed as a Git repository with one-click scripts:

1. **Setup (once):** setup.bat on Windows or ./setup.sh on macOS/Linux creates a Python virtual environment, installs the dependencies (the CPU build of PyTorch on Linux machines without an NVIDIA GPU) and pre-downloads all four models ( $\approx 4$  GB).
2. **Start:** start.bat or ./start.sh launches Uvicorn on http://127.0.0.1:8000 and opens the browser. With --host 0.0.0.0 phones on the same Wi-Fi can connect.
3. **Offline networks:** if huggingface.co is blocked, scripts/convert\_original\_blip.py builds the models from Salesforce's original checkpoints, and .env points the application to the local folders.
4. **Configuration:** .env.example documents every option, such as the default model, device, feature toggles, thread count, host, port and limits.

# CHAPTER 7

## RESULT ANALYSIS

### 7.1 Introduction

This chapter evaluates PixelProse quantitatively and qualitatively. We describe the experimental setup, report caption quality with BLEU, ROUGE-L and CIDEr-D for five pipeline configurations, analyse latency, present qualitative captions, question-answering and explainability results, summarise the engineering findings made during testing, and compare the system with published models and existing captioning tools.

### 7.2 Experimental Setup

**Table 7.1:** Experimental setup

| Parameter         | Value                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------|
| Test set          | 44 images in 6 categories (animals 14, people 9, nature 8, vehicles 6, food 5, art 2)   |
| References        | 3 hand-written captions per image (132 in total), sample_images/references.json         |
| Metrics           | BLEU-1...4, ROUGE-L, CIDEr-D (own implementation, unit-tested), mean ITM match, latency |
| Hardware          | 4-core x86-64 CPU, 16 GB RAM, no GPU; PyTorch with 3 intra-op threads                   |
| Precision         | float32                                                                                 |
| Models            | BLIP-large and BLIP-base captioners, BLIP-ITM (COCO), BLIP-VQA                          |
| Decoding defaults | max 30 new tokens, min 5, repetition penalty 1.1, no-repeat 3-gram, top-p 0.9, T 0.9    |
| Protocol          | Models pre-loaded; one warm-up image per configuration; wall-clock time per image       |

The images span people, animals, food, vehicles, landscapes and paintings, and they were not used to tune any model. Because the ITM match is measured with the same model that performs re-ranking, it is partly self-referential; CIDEr-D, BLEU and ROUGE-L against human references are the independent measures.

### 7.3 Performance Measures

#### 7.3.1 Caption Quality

Table 7.2 reports all metrics for the five configurations, from a naive baseline (BLIP-base with greedy decoding) to the two PixelProse modes.

**Table 7.2:** Caption quality and speed of the evaluated configurations

| Configuration                                   | BLEU-1 | BLEU-4 | ROUGE-L | CIDEr-D | ITM match | sec/img |
|-------------------------------------------------|--------|--------|---------|---------|-----------|---------|
| BLIP-base, greedy                               | 0.742  | 0.351  | 0.605   | 1.819   | 93.7 %    | 1.0     |
| BLIP-large, greedy                              | 0.751  | 0.326  | 0.603   | 1.757   | 88.3 %    | 2.2     |
| BLIP-large, beam search (k = 5)                 | 0.775  | 0.416  | 0.641   | 1.957   | 88.2 %    | 3.2     |
| PixelProse Balanced (5 beams + ITM re-rank)     | 0.771  | 0.408  | 0.626   | 1.945   | 92.1 %    | 3.9     |
| PixelProse Detailed (beam + sampling + re-rank) | 0.771  | 0.399  | 0.625   | 1.913   | 97.0 %    | 5.6     |

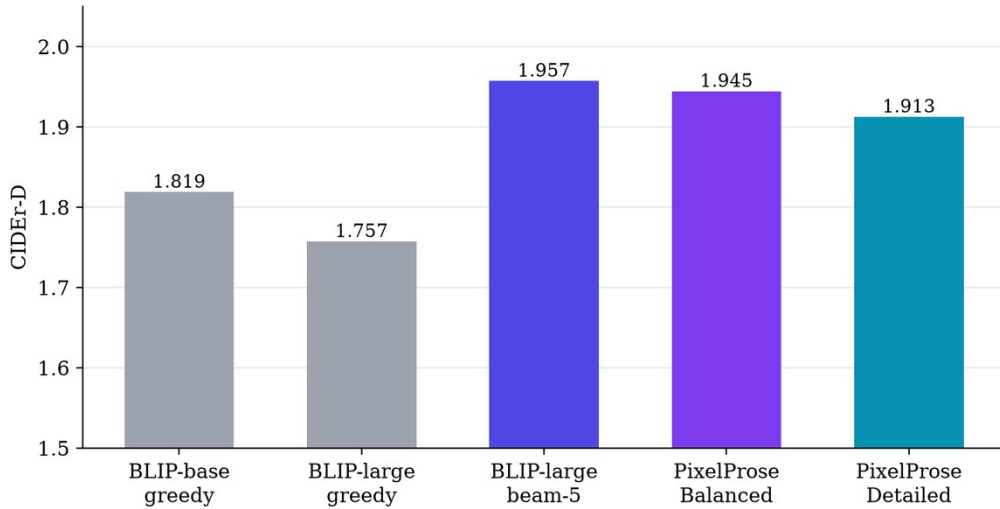
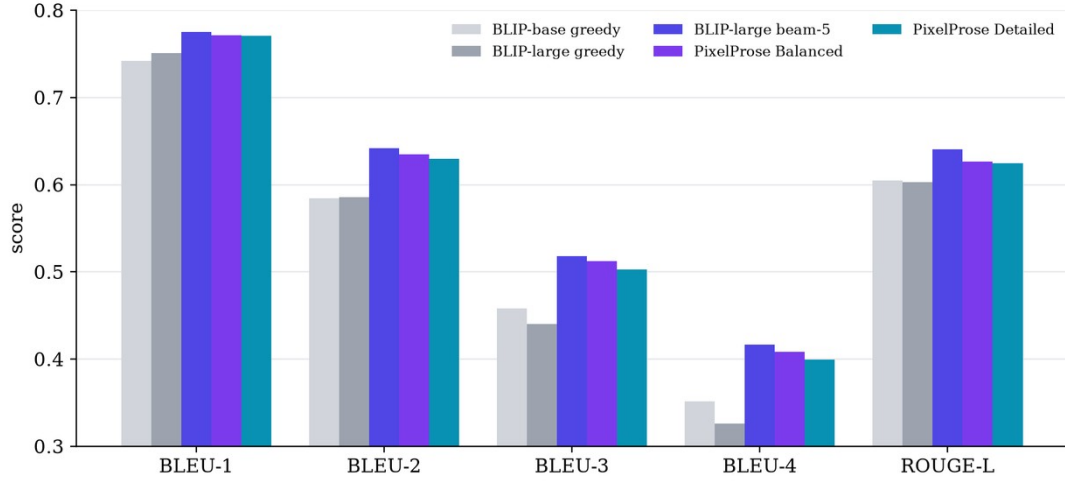
**Figure 7.1:** CIDEr-D of the evaluated configurations

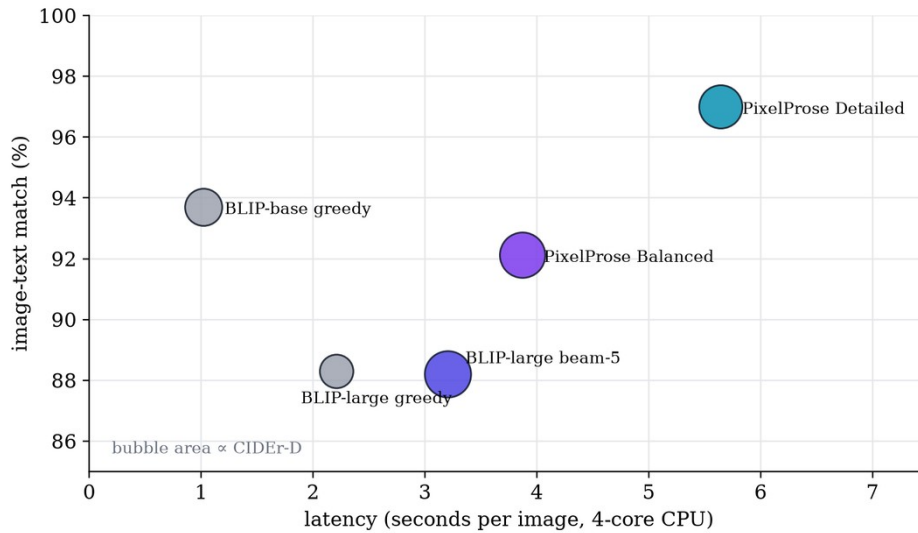
Figure 7.1 and Figure 7.2 visualise the results. Three observations stand out. **First, decoding matters more than model size:** BLIP-large with greedy decoding scores *below* BLIP-base with greedy decoding (CIDEr-D 1.757 vs 1.819). Beam search unlocks the larger model and gives the best reference-based scores, a 7.6 % CIDEr-D gain over the baseline and the highest BLEU at every n-gram order. **Second, ITM re-ranking buys grounding almost for free:** choosing among the five beams with the ITM model keeps CIDEr-D within 0.6 % of plain beam search (1.945 vs 1.957) while raising the image-match from 88.2 % to 92.1 %. It is also deterministic, which makes results reproducible and cacheable, so it is the default *Balanced* mode. **Third, the Detailed mode** reaches the highest image-match (97.0 %) with richer, more specific captions, but the short references do not contain those extra details, so its n-gram scores drop slightly.



**Figure 7.2:** BLEU-1...4 and ROUGE-L of the evaluated configurations

### 7.3.2 Quality–Speed Trade-off

Figure 7.3 plots image-match against latency, with bubble area proportional to CIDEr-D. BLIP-base answers in about 1.0 s and is the right choice for older laptops. The Balanced mode sits at about 3.9 s with high CIDEr-D and 92 % match, and the Detailed mode trades about 1.8 extra seconds for the highest grounding. All configurations meet the non-functional requirement of under 5 seconds per caption on a laptop CPU, except the optional Detailed mode.



**Figure 7.3:** Image–text match versus latency (bubble area  $\propto$  CIDEr-D)

### 7.3.3 Latency Breakdown

Table 7.3 breaks down the Balanced pipeline on five sample images, as reported in the interface's *Pipeline timing* panel. The ViT encoding and beam decoding take roughly

equal time. The batched scoring pass costs under half a second thanks to batching, and ITM re-ranking adds about one second.

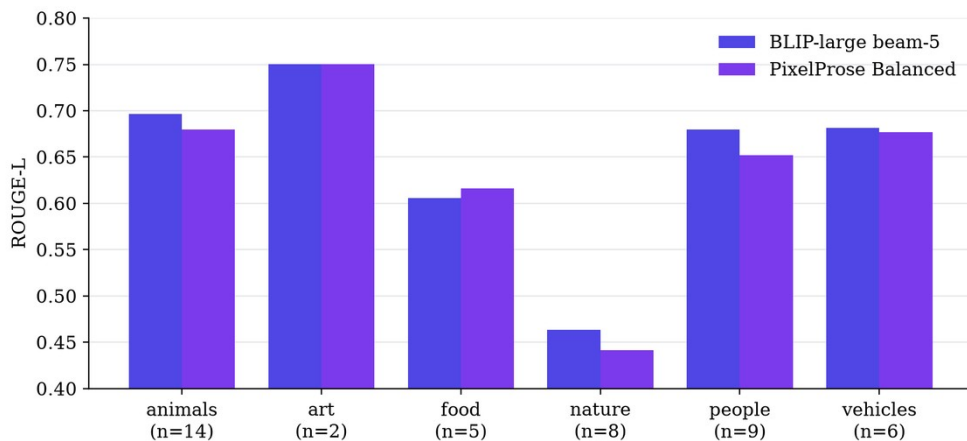
**Table 7.3:** Latency of each pipeline stage in Balanced mode (4-core CPU)

| Image                  | Vision (ms) | Decode (ms) | Scoring (ms) | Re-rank (ms) | Total (ms) |
|------------------------|-------------|-------------|--------------|--------------|------------|
| woman_and_dog_on_beach | 1450        | 1158        | 438          | 871          | 3917       |
| two_cats_sleeping      | 1413        | 1130        | 417          | 1654         | 4614       |
| street_with_bus        | 1366        | 1311        | 446          | 968          | 4091       |
| pizza                  | 1183        | 1103        | 540          | 1447         | 4274       |
| red_sports_car         | 1334        | 1148        | 426          | 1028         | 3936       |
| Mean                   | 1349        | 1170        | 453          | 1194         | 4166       |

Repeated requests for the same image and settings are served from the result cache in a few milliseconds, and switching to BLIP-base or a CUDA GPU reduces latency further.

### 7.3.4 Performance by Category

Figure 7.4 shows ROUGE-L by image category. Artworks, animals, vehicles and people are described well. Nature scenes score lowest because landscapes admit many equally valid descriptions (“a mountain valley”, “a green valley surrounded by mountains”, “a scenic view”), so word overlap with any one reference is naturally low. This illustrates a known limitation of reference-based metrics rather than a failure of the model.

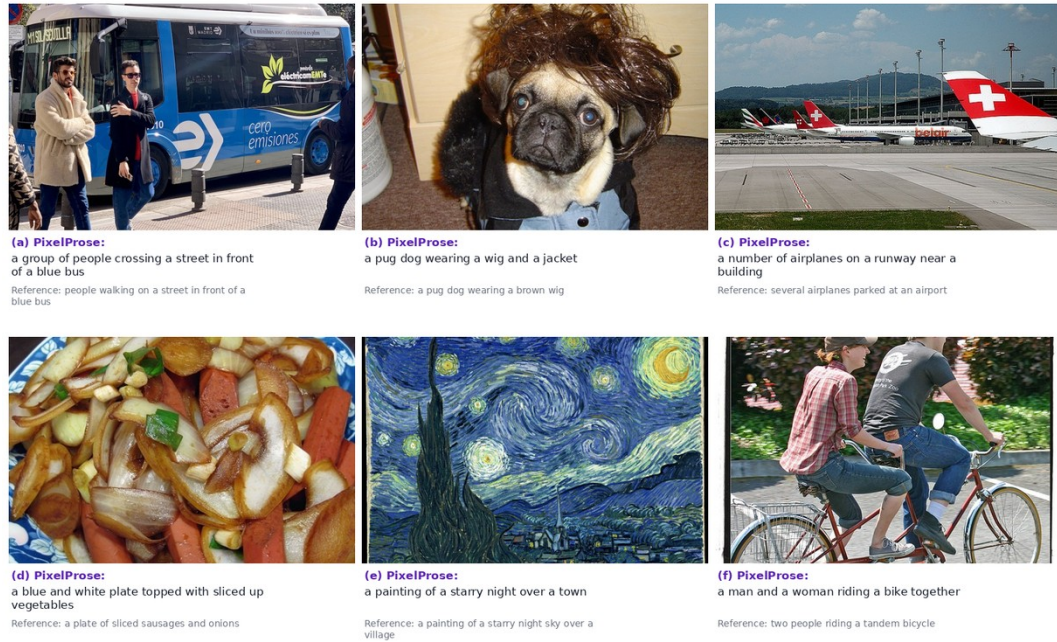


**Figure 7.4:** ROUGE-L by image category

## 7.4 Qualitative Results

Figure 7.5 shows captions produced by the default Balanced mode for six test images, with one human reference each. The captions are accurate, fluent and often more

specific than the references: the model notices the *blue* bus, the pug's wig *and* jacket, and the *blue and white* plate.



**Figure 7.5:** Qualitative results of PixelProse (Balanced mode) with human references

**Table 7.4:** Captions produced by different configurations

| Image                | BLIP-base greedy                                          | BLIP-large beam-5                                     | PixelProse Balanced                                        | PixelProse Detailed                                            |
|----------------------|-----------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|
| airliners_at_airport | a row of airplanes parked on the tarmac                   | a number of airplanes on a runway                     | a number of airplanes on a runway near a building          | a bunch of red and white airplanes parked on an airport tarmac |
| street_with_buses    | people are standing on the sidewalk near a bus            | a group of people crossing a street in front of a bus | a group of people crossing a street in front of a blue bus | a group of people crossing a street in front of a blue bus     |
| stir_fry_plate       | a plate of food with onions, carrots and other vegetables | a blue and white plate topped with lots of food       | a blue and white plate topped with sliced up vegetables    | a blue and white plate topped with sliced up vegetables        |
| pug_in_wig           | a dog with a wig and a jacket on                          | a pug dog wearing a wig and a jacket                  | a pug dog wearing a wig and a jacket                       | a pug dog wearing a wig and a jacket                           |
| the_scream_painting  | a painting of a person with a scream on it                | a painting of two people standing on a bridge         | a painting of two people standing on a bridge              | an edvardt is the scream, painting by edvard stamten           |

Table 7.4 compares configurations on the same images. Re-ranking corrects vague beam outputs: “a number of airplanes on a runway” becomes “a number of airplanes on a runway near a building”, and “topped with lots of food” becomes “topped with sliced up vegetables”. The last row shows the risk of sampling: in Detailed mode the model recognised *The Scream* but produced misspelled names (“an edvardt is the scream, painting by edvard stamten”).

painting by edvard stamten”), which the matcher still scored highly because the painting really is *The Scream*. This failure case motivated making the deterministic Balanced mode the default.

## 7.5 Visual Question Answering Results

**Table 7.5:** Sample visual question answering results

| Image                  | Question                    | Answer            | Confidence |
|------------------------|-----------------------------|-------------------|------------|
| woman_and_dog_on_beach | What is the woman wearing?  | plaid shirt       | 56.4 %     |
| woman_and_dog_on_beach | Where was this taken?       | beach             | 67.2 %     |
| woman_and_dog_on_beach | Is the dog happy?           | yes               | 84.7 %     |
| woman_and_dog_on_beach | What animal is this?        | dog               | 89.3 %     |
| two_cats_sleeping      | How many cats are there?    | 2                 | 89.5 %     |
| two_cats_sleeping      | What color is the blanket?  | pink              | 87.0 %     |
| two_cats_sleeping      | Are the cats sleeping?      | yes               | 88.1 %     |
| street_with_bus        | What color is the bus?      | blue              | 71.8 %     |
| pizza                  | What food is this?          | pizza             | 90.7 %     |
| pizza                  | Is there cheese on it?      | yes               | 89.6 %     |
| red_sports_car         | What color is the car?      | red               | 87.8 %     |
| giraffe_and_zebra      | What animals are there?     | giraffe and zebra | —          |
| school_bus             | How many animals are there? | 0                 | —          |

Table 7.5 lists answers produced by the Ask feature. All answers shown are correct. The model handles colours, counting, yes/no questions, locations and object identity, and correctly answers “0” when asked about animals in a picture that contains none. Each answer takes 0.5–0.9 s. The least confident answers concern subjective or fine-grained attributes (clothing, time of day), and counting larger groups is less reliable: for the street scene “how many people are there?” returned “5” with only 39 % confidence.

## 7.6 Explainability Results

The Grad-CAM heatmaps (Figure 6.4) consistently localise content words: nouns such as *dog*, *woman*, *pizza* or *bus* highlight the corresponding objects, while function words (“a”, “on”, “of”) produce diffuse maps and are displayed in a muted style. Computing the maps for a full caption takes about 0.5 s on the CPU. The per-word probability chart complements the maps. In the beach example, “sitting” (25 %) and “next” (21 %) are the least certain words, which matches intuition: the woman is sitting, but the dog could equally be described as “with” rather than “next to” her. The Match tab gives a further

diagnostic. A correct description scores 99.3 %, a partially correct one (“a dog giving a high five”) 29.1 % and unrelated ones 0.0 %, showing that the matcher reasons about the whole scene rather than single keywords.

## 7.7 Engineering Findings

Testing surfaced three issues whose diagnosis improved the system significantly (Table 7.6).

**Table 7.6:** Engineering issues found during testing

| Issue                                                             | Root cause                                                                            | Fix                                                                               | Effect                                                      |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------|
| Correct captions received a 2 % ITM match                         | Text config used 8 attention heads instead of BLIP’s 12; weights loaded without error | Set num_attention_heads = 12; validate against reference outputs                  | Match rose to 99.6 %, equal to the official reference value |
| A caption took > 90 s in the browser vs 5 s from the command line | PyTorch threads spin-wait; browser rendering occupied one of four cores               | Use cores – 1 threads, serialise CPU inference, pause animations during inference | ≈ 8 s end-to-end in a headless browser, > 10× faster        |
| Nonsense words such as “arafed” in captions                       | Unconditional decoding of BLIP-large (trained with a prompt)                          | Condition on “a picture of” and clean known artefacts                             | Clean, fluent captions                                      |

## 7.8 Comparative Analysis

### 7.8.1 Comparison with Published Models

Table 7.7 places BLIP, the model family used by PixelProse, among the captioning models reviewed in Chapter 2, using the BLEU-4 and CIDEr scores reported by their authors on the MS-COCO Karpathy test split. BLIP-large improves BLEU-4 by 12.7 points over the original Show-and-Tell model and is competitive with much larger systems. BLIP-2 scores higher but uses billions of parameters and generally requires a GPU, which conflicts with our requirement to run on a laptop CPU. The COCO numbers are computed on 5,000 images with five references each and CIDEr scaled by 100, so they are not directly comparable with the scores on our 44-image set.

**Table 7.7:** Published results on MS-COCO (Karpathy test split) as reported by the authors

| Model                            | Year | Encoder / decoder           | BLEU-4 | CIDEr | Runs on laptop CPU |
|----------------------------------|------|-----------------------------|--------|-------|--------------------|
| Show and Tell (NIC) [6]          | 2015 | CNN / LSTM                  | 27.7   | —     | Yes (small)        |
| Show, Attend and Tell (soft) [9] | 2015 | CNN grid + attention / LSTM | 24.3   | —     | Yes                |
| Bottom-Up Top-Down [12]          | 2018 | Faster R-CNN / LSTM         | 36.3   | 120.1 | Slow (detector)    |
| M <sup>2</sup> Transformer [14]  | 2020 | Regions / Transformer       | 39.1   | 131.2 | Slow (detector)    |

| Model                       | Year | Encoder / decoder      | BLEU-4 | CIDEr | Runs on laptop CPU   |
|-----------------------------|------|------------------------|--------|-------|----------------------|
| Oscar-L [16]                | 2020 | Regions + tags / BERT  | 41.7   | 140.0 | Slow (detector)      |
| BLIP ViT-L (used here) [19] | 2022 | ViT-L / BERT decoder   | 40.4   | 136.7 | Yes ( $\approx 4$ s) |
| BLIP-2 [21]                 | 2023 | ViT-g + Q-Former / LLM | 43.7   | 145.8 | No (GPU)             |

### 7.8.2 Comparison with Existing Systems

Table 7.8 compares PixelProse with typical kinds of existing captioning solutions: cloud vision APIs from major providers, online research demos of captioning models, and a basic single-call script that uses a pre-trained model.

**Table 7.8:** Feature comparison with existing captioning solutions

| Feature                          | Cloud vision API | Online model demo   | Basic script | PixelProse     |
|----------------------------------|------------------|---------------------|--------------|----------------|
| Works offline / private          | No               | No                  | Yes          | Yes            |
| Cost per image                   | Paid             | Free (rate-limited) | Free         | Free           |
| Multiple candidates with scores  | Limited          | No                  | No           | Yes            |
| Independent verification (ITM)   | No               | No                  | No           | Yes            |
| Word-level visual explanation    | No               | No                  | No           | Yes (Grad-CAM) |
| Per-word probabilities           | No               | No                  | No           | Yes            |
| Visual question answering        | Separate service | Separate demo       | No           | Integrated     |
| Custom description matching      | No               | No                  | No           | Yes            |
| Batch + CSV/JSON export          | Via code         | No                  | Via code     | Yes (UI)       |
| Accessibility (alt-text, speech) | Alt-text only    | No                  | No           | Yes            |
| REST API + CLI                   | API only         | No                  | No           | Both           |
| Measured quality (BLEU/CIDEr)    | Not disclosed    | No                  | No           | Yes            |

PixelProse is not intended to beat the largest cloud models in raw accuracy. Its contribution is a **complete, explainable, verifiable and private** captioning system that achieves strong measured quality on commodity hardware and makes the internal reasoning of a vision–language transformer visible to its users.

## 7.9 Limitations

The evaluation set of 44 images is small, so the numbers show trends rather than leaderboard results. Captions are single English sentences; dense or paragraph-length descriptions are not produced. The model can miscount larger groups and occasionally

misses fine details or proper names such as artworks and landmarks. Being trained on web data, it may reflect social biases present in that data. Finally, about 5 GB of RAM and 4 GB of disk are needed for all four models, which excludes very low-end devices.

## CHAPTER 8

### CONCLUSION & FUTURE SCOPE

#### 8.1 Conclusion

In this project we designed, implemented and evaluated **PixelProse**, an explainable image caption generator built on the BLIP family of vision–language transformers. Starting from a survey of captioning research, from template systems through CNN–LSTM encoder–decoders and attention models to large-scale vision–language pre-training, we identified gaps between research models and usable tools: captions are rarely verified or explained, interaction is limited and the strongest models depend on GPUs or paid cloud services.

PixelProse addresses these gaps with an engineered pipeline around pre-trained BLIP models. The image is encoded once by a Vision Transformer. Several candidate captions are decoded with beam search and, optionally, nucleus sampling. Every candidate is scored by its token probabilities, and an independent image–text matching model verifies and re-ranks the candidates. Grad-CAM on cross-attention makes the model's reasoning visible word by word, a visual question-answering model lets users interrogate the image, and a matching tool lets them test their own descriptions. The system is delivered as an animated, responsive and accessible web application with five input methods, batch processing, history, text-to-speech and shareable cards, together with a documented REST API and a command-line tool.

We evaluated five configurations on 44 images with 132 hand-written references using our own implementations of BLEU, ROUGE-L and CIDEr-D. Beam search with the large model improved CIDEr-D by 7.6 % over a greedy baseline. The default Balanced mode (beam search with ITM re-ranking) matched that quality (CIDEr-D 1.945) while raising the image–text match from 88 % to 92 %, at about 3.9 seconds per image on a 4-core CPU. The Detailed mode reached 97 % match with richer captions. Visual question answering correctly handled colours, counts, objects and yes/no questions, and the heatmaps consistently localised the objects named in the captions. Along the way, systematic testing uncovered and fixed a silent configuration bug and a CPU thread-contention problem, reinforcing the importance of validating AI systems against known outputs and measuring them end to end.

All the objectives set out in Section 1.3 were met. PixelProse demonstrates that a modern, explainable and verifiable captioning system can run privately on an ordinary laptop, and it serves as an effective educational tool for understanding how attention-based vision–language models see and describe the world.

## 8.2 Future Scope

- **Multilingual captions:** add Indian languages such as Hindi, Marathi and Gujarati through a translation model or a multilingual captioner, extending accessibility to non-English speakers.
- **Larger vision–language models:** offer BLIP-2, Florence-2 or LLaVA-style models as an optional GPU mode for paragraph-length, dense descriptions.
- **Fine-tuning on local data:** fine-tune the captioner on Indian scenes or on domain data (medical imaging, retail products, agriculture) to reduce the Western bias of web training data.
- **Video captioning:** describe video clips scene by scene by sampling frames and adding temporal modelling.
- **On-device deployment:** quantise the models (INT8) and export them to ONNX or TensorFlow Lite for a mobile application usable by visually impaired users without a computer.
- **Reinforcement-learning optimisation:** apply self-critical sequence training with CIDEr and ITM rewards to directly optimise caption quality and grounding.
- **Larger evaluation:** evaluate on the full MS-COCO Karpathy test split and with human raters, and add metrics such as SPICE and CLIPScore.
- **Bias and safety auditing:** measure and mitigate demographic bias and add filters for sensitive content.
- **Integrations:** browser extensions and CMS plug-ins (e.g., WordPress) that generate alt-text automatically when images are uploaded.

## REFERENCES

- [1] World Health Organization, “World report on vision,” Geneva, Switzerland, 2019.
- [2] WebAIM, “The WebAIM Million: An annual accessibility analysis of the top 1,000,000 home pages,” Utah State University. [Online]. Available: <https://webaim.org/projects/million/>
- [3] M. Z. Hossain, F. Sohel, M. F. Shiratuddin and H. Laga, “A comprehensive survey of deep learning for image captioning,” *ACM Computing Surveys*, vol. 51, no. 6, pp. 1–36, 2019.
- [4] M. Stefanini, M. Cornia, L. Baraldi, S. Cascianelli, G. Fiameni and R. Cucchiara, “From show to tell: A survey on deep learning-based image captioning,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 539–559, 2023.
- [5] G. Kulkarni, V. Premraj, S. Dhar, S. Li, Y. Choi, A. C. Berg and T. L. Berg, “Baby talk: Understanding and generating simple image descriptions,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 1601–1608.
- [6] O. Vinyals, A. Toshev, S. Bengio and D. Erhan, “Show and tell: A neural image caption generator,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3156–3164.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.
- [8] A. Karpathy and L. Fei-Fei, “Deep visual-semantic alignments for generating image descriptions,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3128–3137.
- [9] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel and Y. Bengio, “Show, attend and tell: Neural image caption generation with visual attention,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 2048–2057.
- [10] J. Lu, C. Xiong, D. Parikh and R. Socher, “Knowing when to look: Adaptive attention via a visual sentinel for image captioning,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 375–383.
- [11] S. J. Rennie, E. Marcheret, Y. Mroueh, J. Ross and V. Goel, “Self-critical sequence training for image captioning,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7008–7024.

- [12] P. Anderson, X. He, C. Buehler, D. Teney, M. Johnson, S. Gould and L. Zhang, “Bottom-up and top-down attention for image captioning and visual question answering,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 6077–6086.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.
- [14] M. Cornia, M. Stefanini, L. Baraldi and R. Cucchiara, “Meshed-memory transformer for image captioning,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020, pp. 10578–10587.
- [15] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby, “An image is worth 16×16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learning Representations (ICLR), 2021.
- [16] X. Li, X. Yin, C. Li, P. Zhang, X. Hu, L. Zhang, L. Wang, H. Hu, L. Dong, F. Wei, Y. Choi and J. Gao, “Oscar: Object-semantics aligned pre-training for vision-language tasks,” in Proc. European Conf. Computer Vision (ECCV), 2020, pp. 121–137.
- [17] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, “Learning transferable visual models from natural language supervision,” in Proc. Int. Conf. Machine Learning (ICML), 2021, pp. 8748–8763.
- [18] J. Li, R. R. Selvaraju, A. D. Gotmare, S. Joty, C. Xiong and S. C. H. Hoi, “Align before fuse: Vision and language representation learning with momentum distillation,” in Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [19] J. Li, D. Li, C. Xiong and S. Hoi, “BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” in Proc. Int. Conf. Machine Learning (ICML), 2022, pp. 12888–12900.
- [20] J. Wang, Z. Yang, X. Hu, L. Li, K. Lin, Z. Gan, Z. Liu, C. Liu and L. Wang, “GIT: A generative image-to-text transformer for vision and language,” Trans. Machine Learning Research (TMLR), 2022.
- [21] J. Li, D. Li, S. Savarese and S. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in Proc. Int. Conf. Machine Learning (ICML), 2023, pp. 19730–19742.
- [22] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based

- localization,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 618–626.
- [23] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick and D. Parikh, “VQA: Visual question answering,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2015, pp. 2425–2433.
  - [24] K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318.
  - [25] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Text Summarization Branches Out (ACL Workshop), 2004, pp. 74–81.
  - [26] R. Vedantam, C. L. Zitnick and D. Parikh, “CIDEr: Consensus-based image description evaluation,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2015, pp. 4566–4575.
  - [27] P. Anderson, B. Fernando, M. Johnson and S. Gould, “SPICE: Semantic propositional image caption evaluation,” in Proc. European Conf. Computer Vision (ECCV), 2016, pp. 382–398.
  - [28] A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi, “The curious case of neural text degeneration,” in Proc. Int. Conf. Learning Representations (ICLR), 2020.
  - [29] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
  - [30] B. W. Boehm, Software Engineering Economics. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981.
  - [31] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.
  - [32] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8024–8035.
  - [33] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38–45.
  - [34] S. Ramírez, “FastAPI: Modern, fast (high-performance) web framework for building APIs with Python,” 2018. [Online]. Available: <https://fastapi.tiangolo.com>

## **PUBLISHED PAPERS**

*(Attach the published / communicated research paper(s) based on this project here.)*

## **CERTIFICATES**

*(Attach paper publication certificates, conferences attended and competitions participated in here.)*

## **PLAGIARISM REPORT**

*(Attach the first few pages of the plagiarism report here.)*